

10 고급 *pandas*





강의 내용

Contents of Lecture

기간	내용	과제
01주차	오리엔테이션 Python 언어의 기본, lpython, 주피터 노트북	-
02주차	내장 자료구조, 함수, 파일	실습 과제
03주차	Numpy 기본 : 배열과 벡터 연산	실습 과제
04주차	Pandas 시작하기	실습 과제
05주차	데이터 로딩과 저장, 파일 형식	실습 과제
06주차	데이터 정제 및 준비(조인, 병합, 변형)	실습 과제
07주차	그래프와 시각화	기말 프로젝트 상세 공지
08주차	중간고사 - 오프라인 오픈북으로 진행	실습 과제

기간	내용	과제
09주차	데이터 집계와 그룹 연산	실습 과제
10주차	시계열	실습 과제
11주차	고급 pandas	실습 과제
12주차	파이썬 모델링 라이브러리	실습 과제
13주차	데이터 분석 예제 / 고급 Numpy	실습 과제
14주차	Kaggle / Dacon 데이터 분석 사례 Case Study	프로젝트 레포트 제출
15주차	[기말고사] Team별 프로젝트 결과 발표	종강~!

1부: *Pandas* 기능 총정리





pandas 기능 총정리

Functions of pandas

Pandas를 배우는 법

> 책 / 수업 / 온라인 검색

Pandas를 활용하는 법

> 용도에 따라 **Documentation**에서 **사용법** 확인하여 적용

(용도가 명확히 구체화가 되지 않을 때는 pandas와 함께 영문으로 하고자 하는 내용을 비슷하게 검색해볼 것!)

(**Stackoverflow** 사이트에는 Global하게 많은 개발자들로부터 발생하는 세세한 Q&A Case들을 기록되어있다!)

따라서, Pandas Documentation에서 사용법을 확인하는 방법에 대해서 알아보자!





pandas 기능 총정리

Functions of pandas

1. Series
2. DataFrame
3. Index Alignment (인덱스 할당)
4. Reindexing
5. Row/Column 삭제
6. Missing Data 처리 (Filtering / Cleaning / Replacing)
7. 수치형 -> 범주화 (Discretization / Binning)
8. Outlier Detection & Filtering
9. Indexing / Slicing / Masking(Filtering) / Selection
10. 정수 Indexing
11. 산술 연산
12. 데이터 정렬
13. 정렬 및 순위
14. 함수 적용 및 매핑
15. 범주형 -> 수치화 (Dummy Variable)
16. 문자열 다루기 (+ Regular Expression)
17. 계층적 인덱싱 (Multilevel-Index)
18. 데이터 합치기 & 조인 (Concat & Merge & Join)
19. 시계열





pandas 기능 총정리

Functions of pandas

20. 기초통계량

- Describe 함수 및 기타 방법
- 상관계수/공분산
- Unique Value / Unique Count / 각 Value별 Count

21. Aggregation & Grouping

- Aggregate
- Groupby

22. Pivot Table

23. 시각화 (plot 함수)

24. 파일 입출력





Series > 기본 구조

pandas.Series

```
class pandas.Series(data=None, index=None, dtype=None, name=None, copy=False, fastpath=False)
```

[\[source\]](#)

One-dimensional ndarray with axis labels (including time series).

Labels need not be unique but must be a hashable type. The object supports both integer- and label-based indexing and provides a host of methods for performing operations involving the index. Statistical methods from ndarray have been overridden to automatically exclude missing data (currently represented as NaN).

Operations between Series (+, -, /, *, **) align values based on their associated index values– they need not be the same length. The result index will be the sorted union of the two indexes.

Parameters: **data** : *array-like, iterable, dict, or scalar value*

Contains data stored in Series. If data is a dict, argument order is maintained.

index : *array-like or Index (1d)*

Values must be hashable and have the same length as *data*. Non-unique index values are allowed. Will default to RangeIndex (0, 1, 2, ..., n) if not provided. If data is dict-like and index is None, then the keys in the data are used as the index. If the index is not None, the resulting Series is reindexed with the index values.

dtype : *str, numpy.dtype, or ExtensionDtype, optional*

Data type for the output Series. If not specified, this will be inferred from *data*. See the [user guide](#) for more usages.

name : *str, optional*

The name to give to the Series.

copy : *bool, default False*

Copy input data. Only affects Series or 1d ndarray input. See examples.

- 기본 활용구조 (Class)

- Input Parameter

- data
- index
- dtype
- name
- copy





Series > 예시

Examples

Constructing Series from a dictionary with an Index specified

```
>>> d = {'a': 1, 'b': 2, 'c': 3}
>>> ser = pd.Series(data=d, index=['a', 'b', 'c'])
>>> ser
a    1
b    2
c    3
dtype: int64
```

The keys of the dictionary match with the Index values, hence the Index values have no effect.

```
>>> d = {'a': 1, 'b': 2, 'c': 3}
>>> ser = pd.Series(data=d, index=['x', 'y', 'z'])
>>> ser
x    NaN
y    NaN
z    NaN
dtype: float64
```

Note that the Index is first build with the keys from the dictionary. After this the Series is reindexed with the given Index values, hence we get all NaN as a result.





Series > Attribute (함수가 아니어서 ()를 사용할 필요 X)

Attributes

<code>T</code>	Return the transpose, which is by definition self.
<code>array</code>	The ExtensionArray of the data backing this Series or Index.
<code>at</code>	Access a single value for a row/column label pair.
<code>attrs</code>	Dictionary of global attributes of this dataset.
<code>axes</code>	Return a list of the row axis labels.
<code>dtype</code>	Return the dtype object of the underlying data.
<code>dtypes</code>	Return the dtype object of the underlying data.
<code>flags</code>	Get the properties associated with this pandas object.
<code>hasnans</code>	Return True if there are any NaNs.
<code>iat</code>	Access a single value for a row/column pair by integer position.
<code>iloc</code>	Purely integer-location based indexing for selection by position.
<code>index</code>	The index (axis labels) of the Series.



Series > Method (함수여서 ()를 사용해야함)

Methods

<code>abs()</code>	Return a Series/DataFrame with absolute numeric value of each element.
<code>add(other[, level, fill_value, axis])</code>	Return Addition of series and other, element-wise (binary operator <i>add</i>).
<code>add_prefix(prefix)</code>	Prefix labels with string <i>prefix</i> .
<code>add_suffix(suffix)</code>	Suffix labels with string <i>suffix</i> .
<code>agg([func, axis])</code>	Aggregate using one or more operations over the specified axis.
<code>aggregate([func, axis])</code>	Aggregate using one or more operations over the specified axis.
<code>align(other[, join, axis, level, copy, ...])</code>	Align two objects on their axes with the specified join method.
<code>all([axis, bool_only, skipna, level])</code>	Return whether all elements are True, potentially over an axis.
<code>any([axis, bool_only, skipna, level])</code>	Return whether any element is True, potentially over an axis.
<code>append(to_append[, ignore_index, ...])</code>	(DEPRECATED) Concatenate two or more Series.
<code>apply(func[, convert_dtype, args])</code>	Invoke function on values of Series.
<code>argmax([axis, skipna])</code>	Return int position of the largest value in the Series.
<code>argmin([axis, skipna])</code>	Return int position of the smallest value in the Series.
<code>argsort([axis, kind, order])</code>	Return the integer indices that would sort the Series values.

- Documentation 확인시,
Attribute에 비해서 Method는 상당히 많은 기능이 존재
(Attribute 약 20개 기능 < Method 약 100개 기능)





DataFrame

pandas.DataFrame

`class pandas.DataFrame(data=None, index=None, columns=None, dtype=None, copy=None)`

Two-dimensional, size-mutable, potentially heterogeneous tabular data.

[\[source\]](#)

Data structure also contains labeled axes (rows and columns). Arithmetic operations align on both row and column labels. Can be thought of as a dict-like container for Series objects. The primary pandas data structure.

Parameters: **data** : ndarray (structured or homogeneous), Iterable, dict, or DataFrame

Dict can contain Series, arrays, constants, dataclass or list-like objects. If data is a dict, column order follows insertion-order. If a dict contains Series which have an index defined, it is aligned by its index.

❗ Changed in version 0.25.0: If data is a list of dicts, column order follows insertion-order.

index : Index or array-like

Index to use for resulting frame. Will default to RangeIndex if no indexing information part of input data and no index provided.

columns : Index or array-like

Column labels to use for resulting frame when data does not have them, defaulting to RangeIndex(0, 1, 2, ..., n). If data contains column labels, will perform column selection instead.

dtype : dtype, default None

Data type to force. Only a single dtype is allowed. If None, infer.

copy : bool or None, default None

Copy data from inputs. For dict data, the default of None behaves like `copy=True`. For DataFrame or 2d ndarray input, the default of None behaves like `copy=False`.

Examples

Constructing DataFrame from a dictionary.

```
>>> d = {'col1': [1, 2], 'col2': [3, 4]}
>>> df = pd.DataFrame(data=d)
>>> df
   col1  col2
0     1     3
1     2     4
```

Notice that the inferred dtype is int64.

```
>>> df.dtypes
col1    int64
col2    int64
dtype: object
```

To enforce a single dtype:

```
>>> df = pd.DataFrame(data=d, dtype=np.int8)
>>> df.dtypes
col1    int8
col2    int8
dtype: object
```

Constructing DataFrame from a dictionary including Series:

```
>>> d = {'col1': [0, 1, 2, 3], 'col2': pd.Series([2, 3], index=[2, 3])}
>>> pd.DataFrame(data=d, index=[0, 1, 2, 3])
   col1  col2
0     0   NaN
1     1   NaN
2     2   2.0
3     3   3.0
```





DataFrame

Attributes

<code>at</code>	Access a single value for a row/column label pair.
<code>attrs</code>	Dictionary of global attributes of this dataset.
<code>axes</code>	Return a list representing the axes of the DataFrame.
<code>columns</code>	The column labels of the DataFrame.
<code>dtypes</code>	Return the dtypes in the DataFrame.
<code>empty</code>	Indicator whether Series/DataFrame is empty.
<code>flags</code>	Get the properties associated with this pandas object.
<code>iat</code>	Access a single value for a row/column pair by integer position.
<code>iloc</code>	Purely integer-location based indexing for selection by position.
<code>index</code>	The index (row labels) of the DataFrame.
<code>loc</code>	Access a group of rows and columns by label(s) or a boolean array.

Methods

<code>abs()</code>	Return a Series/DataFrame with absolute numeric each element.
<code>add(other[, axis, level, fill_value])</code>	Get Addition of dataframe and other, element-wise operator <i>add</i>).
<code>add_prefix(prefix)</code>	Prefix labels with string <i>prefix</i> .
<code>add_suffix(suffix)</code>	Suffix labels with string <i>suffix</i> .
<code>agg([func, axis])</code>	Aggregate using one or more operations over the specified axis.
<code>aggregate([func, axis])</code>	Aggregate using one or more operations over the specified axis.
<code>align(other[, join, axis, level, copy, ...])</code>	Align two objects on their axes with the specified method.
<code>all([axis, bool_only, skipna, level])</code>	Return whether all elements are True, potentially on axis.
<code>any([axis, bool_only, skipna, level])</code>	Return whether any element is True, potentially on axis.
<code>append(other[, ignore_index, ...])</code>	(DEPRECATED) Append rows of <i>other</i> to the end of returning a new object.





Index Alignment

pandas.DataFrame

`class pandas.DataFrame(data=None, index=None, columns=None, dtype=None, copy=None)`

Two-dimensional, size-mutable, potentially heterogeneous tabular data.

[\[source\]](#)

Data structure also contains labeled axes (rows and columns). Arithmetic operations align on both row and column labels. Can be thought of as a dict-like container for Series objects. The primary pandas data structure.

Parameters: **data** : ndarray (structured or homogeneous), Iterable, dict, or DataFrame

Dict can contain Series, arrays, constants, dataclass or list-like objects. If data is a dict, column order follows insertion-order. If a dict contains Series which have an index defined, it is aligned by its index.

Changed in version 0.25.0: If data is a list of dicts, column order follows insertion-order.

index : Index or array-like

Index to use for resulting frame. Will default to RangeIndex if no indexing information part of input data and no index provided.

columns : Index or array-like

Column labels to use for resulting frame when data does not have them, defaulting to RangeIndex(0, 1, 2, ..., n). If data contains column labels, will perform column selection instead.

dtype : dtype, default None

Data type to force. Only a single dtype is allowed. If None, infer.

copy : bool or None, default None

Copy data from inputs. For dict data, the default of None behaves like `copy=True`. For DataFrame or 2d ndarray input, the default of None behaves like `copy=False`.

- index : row index 설정
- columns : column index 설정

* 더 세부적인 세팅 방법은 아래 Stackoverflow 링크 참조

<https://stackoverflow.com/questions/18022845/pandas-index-column-title-or-name>





Reindexing

pandas.DataFrame.reindex

`DataFrame.reindex(labels=None, index=None, columns=None, axis=None, method=None, copy=True, level=None, fill_value=nan, limit=None, tolerance=None)` [\[source\]](#)

Conform Series/DataFrame to new index with optional filling logic.

Places NA/NaN in locations having no value in the previous index. A new object is produced unless the new index is equivalent to the current one and `copy=False`.

Parameters: **keywords for axes :** *array-like, optional*

New labels / index to conform to, should be specified using keywords. Preferably an Index object to avoid duplicating data.

method : *{None, 'backfill'/'bfill', 'pad'/'ffill', 'nearest'}*

Method to use for filling holes in reindexed DataFrame. Please note: this is only applicable to DataFrames/Series with a monotonically increasing/decreasing index.

- None (default): don't fill gaps
- pad / ffill: Propagate last valid observation forward to next valid.
- backfill / bfill: Use next valid observation to fill gap.
- nearest: Use nearest valid observations to fill gap.

copy : *bool, default True*

Return a new object, even if the passed indexes are the same.

level : *int or name*

Broadcast across a level, matching Index values on the passed MultiIndex level.

fill_value : *scalar, default np.NaN*

Value to use for missing values. Defaults to NaN, but can be any "compatible" value.

limit : *int, default None*

Maximum number of consecutive elements to forward or backward fill.

tolerance : *optional*

Maximum distance between original and new labels for inexact matches. The values of the index at the matching locations must satisfy the equation $\text{abs}(\text{index}[\text{indexer}] - \text{target}) \leq \text{tolerance}$.

Tolerance may be a scalar value, which applies the same tolerance to all values, or list-like, which applies variable tolerance per element. List-like includes list, tuple, array, Series, and must be the same size as the index and its dtype must exactly match the index's type.

Returns: **Series/DataFrame with changed index.**





Reindexing

Examples

`DataFrame.reindex` supports two calling conventions

- `(index=index_labels, columns=column_labels, ...)`
- `(labels, axis={'index', 'columns'}, ...)`

We *highly* recommend using keyword arguments to clarify your intent.

Create a dataframe with some fictional data.

```
>>> index = ['Firefox', 'Chrome', 'Safari', 'IE10', 'Konqueror']
>>> df = pd.DataFrame({'http_status': [200, 200, 404, 404, 301],
...                     'response_time': [0.04, 0.02, 0.07, 0.08, 1.0]},
...                     index=index)
>>> df
```

	http_status	response_time
Firefox	200	0.04
Chrome	200	0.02
Safari	404	0.07
IE10	404	0.08
Konqueror	301	1.00

Create a new index and reindex the dataframe. By default values in the new index that do not have corresponding records in the dataframe are assigned `NaN`.

```
>>> new_index = ['Safari', 'Iceweasel', 'Comodo Dragon', 'IE10',
...               'Chrome']
>>> df.reindex(new_index)
```

	http_status	response_time
Safari	404.0	0.07
Iceweasel	NaN	NaN
Comodo Dragon	NaN	NaN
IE10	404.0	0.08
Chrome	200.0	0.02



Row/Column 삭제 (Drop)

pandas.DataFrame.drop

`DataFrame.drop(labels=None, axis=0, index=None, columns=None, level=None, inplace=False, errors='raise')` [\[source\]](#)

Drop specified labels from rows or columns.

Remove rows or columns by specifying label names and corresponding axis, or by specifying directly index or column names. When using a multi-index, labels on different levels can be removed by specifying the level. See the *user guide* <[advanced.shown_levels](#)> for more information about the now unused levels.

Parameters: **labels** : *single label or list-like*

Index or column labels to drop. A tuple will be used as a single label and not treated as a list-like.

axis : {0 or 'index', 1 or 'columns'}, default 0

Whether to drop labels from the index (0 or 'index') or columns (1 or 'columns').

index : *single label or list-like*

Alternative to specifying axis (`labels`, `axis=0` is equivalent to `index=labels`).

columns : *single label or list-like*

Alternative to specifying axis (`labels`, `axis=1` is equivalent to `columns=labels`).

level : *int or level name, optional*

For MultiIndex, level from which the labels will be removed.

inplace : *bool, default False*

If False, return a copy. Otherwise, do operation inplace and return None.

errors : {'ignore', 'raise'}, default 'raise'

If 'ignore', suppress error and only existing labels are dropped.

Returns: **DataFrame or None**

DataFrame without the removed index or column labels or None if `inplace=True`.

Raises: **KeyError**

If any of the labels is not found in the selected axis.

Examples

```
>>> df = pd.DataFrame(np.arange(12).reshape(3, 4),
...                    columns=['A', 'B', 'C', 'D'])
>>> df
   A  B  C  D
0  0  1  2  3
1  4  5  6  7
2  8  9 10 11
```

Drop columns

```
>>> df.drop(['B', 'C'], axis=1)
   A  D
0  0  3
1  4  7
2  8 11
```

```
>>> df.drop(columns=['B', 'C'])
   A  D
0  0  3
1  4  7
2  8 11
```

Drop a row by index

```
>>> df.drop([0, 1])
   A  B  C  D
2  8  9 10 11
```




Missing Data 처리 > Filtering

Filtering

- isna() method (= isnull() method)

Cleaning

- dropna() / fillna() method

Replacing : null data가 'nan', '9999', 'n/a'와 같이 문자열로 저장되어있을 경우 None 혹은 nan으로 인식되도록 변경 필요

- replace() method





Missing Data 처리 > Filtering

pandas.isnull

pandas.isnull(obj)

[\[source\]](#)

Detect missing values for an array-like object.

This function takes a scalar or array-like object and indicates whether values are missing (NaN in numeric arrays, None or NaN in object arrays, NaT in datetimelike).

Parameters: **obj** : scalar or array-like

Object to check for null or missing values.

Returns: bool or array-like of bool

For scalar input, returns a scalar boolean. For array input, returns an array of boolean indicating whether each corresponding element is missing.

See also

notna

Boolean inverse of pandas.isna.

Series.isna

Detect missing values in a Series.

DataFrame.isna

Detect missing values in a DataFrame.

Index.isna

Detect missing values in an Index.

For Series and DataFrame, the same type is returned, containing booleans.

```
>>> df = pd.DataFrame([[ 'ant', 'bee', 'cat'], [ 'dog', None, 'fly']])
>>> df
   0    1    2
0  ant  bee  cat
1  dog  None fly
>>> pd.isna(df)
   0    1    2
0 False False False
1 False  True  False
```

```
>>> pd.isna(df[1])
0    False
1     True
Name: 1, dtype: bool
```



Missing Data 처리 > Filtering

`DataFrame.isna()`

[\[source\]](#)

Detect missing values.

Return a boolean same-sized object indicating if the values are NA. NA values, such as `None` or `numpy.NaN`, gets mapped to `True` values. Everything else gets mapped to `False` values. Characters such as empty strings `''` or `numpy.inf` are not considered NA values (unless you set `pandas.options.mode.use_inf_as_na = True`).

Returns: `DataFrame`

Mask of bool values for each element in `DataFrame` that indicates whether an element is an NA value.

See also

`DataFrame.isnull`

Alias of `isna`.

`DataFrame.notna`

Boolean inverse of `isna`.

`DataFrame.dropna`

Omit axes labels with missing values.

`isna`

Top-level `isna`.

`pandas.DataFrame.isnull`

`DataFrame.isnull()`

[\[source\]](#)

`DataFrame.isnull` is an alias for `DataFrame.isna`.

Detect missing values.

Return a boolean same-sized object indicating if the values are NA. NA values, such as `None` or `numpy.NaN`, gets mapped to `True` values. Everything else gets mapped to `False` values. Characters such as empty strings `''` or `numpy.inf` are not considered NA values (unless you set `pandas.options.mode.use_inf_as_na = True`).

Returns: `DataFrame`

Mask of bool values for each element in `DataFrame` that indicates whether an element is an NA value.

See also

`DataFrame.isnull`

Alias of `isna`.

`DataFrame.notna`

Boolean inverse of `isna`.

`DataFrame.dropna`

Omit axes labels with missing values.

`isna`

Top-level `isna`.



Missing Data 처리 (Filtering / Cleaning / Replacing)

`DataFrame.isna()`

[\[source\]](#)

Detect missing values.

Return a boolean same-sized object indicating if the values are NA. NA values, such as `None` or `numpy.NaN`, gets mapped to `True` values. Everything else gets mapped to `False` values. Characters such as empty strings `''` or `numpy.inf` are not considered NA values (unless you set `pandas.options.mode.use_inf_as_na = True`).

Returns: DataFrame

Mask of bool values for each element in DataFrame that indicates whether an element is an NA value.

See also

`DataFrame.isnull`

Alias of `isna`.

`DataFrame.notna`

Boolean inverse of `isna`.

`DataFrame.dropna`

Omit axes labels with missing values.

`isna`

Top-level `isna`.

Examples

Show which entries in a DataFrame are NA.

```
>>> df = pd.DataFrame(dict(age=[5, 6, np.NaN],
...                        born=[pd.NaT, pd.Timestamp('1939-05-27'),
...                             pd.Timestamp('1940-04-25')],
...                        name=['Alfred', 'Batman', ''],
...                        toy=[None, 'Batmobile', 'Joker']))
>>> df
```

	age	born	name	toy
0	5.0	NaT	Alfred	None
1	6.0	1939-05-27	Batman	Batmobile
2	NaN	1940-04-25		Joker

```
>>> df.isna()
```

	age	born	name	toy
0	False	True	False	True
1	False	False	False	False
2	True	False	False	False



Missing Data 처리 > Cleaning

pandas.DataFrame.dropna

`DataFrame.dropna(axis=0, how='any', thresh=None, subset=None, inplace=False)` [\[source\]](#)

Remove missing values.

See the [User Guide](#) for more on which values are considered missing, and how to work with missing data.

Parameters: **axis** : {0 or 'index', 1 or 'columns'}, default 0

Determine if rows or columns which contain missing values are removed.

- 0, or 'index' : Drop rows which contain missing values.
- 1, or 'columns' : Drop columns which contain missing value.

Changed in version 1.0.0: Pass tuple or list to drop on multiple axes. Only a single axis is allowed.

how : {'any', 'all'}, default 'any'

Determine if row or column is removed from DataFrame, when we have at least one NA or all NA.

- 'any' : If any NA values are present, drop that row or column.
- 'all' : If all values are NA, drop that row or column.

thresh : int, optional

Require that many non-NA values.

subset : column label or sequence of labels, optional

Labels along other axis to consider, e.g. if you are dropping rows these would be a list of columns to include.

inplace : bool, default False

If True, do operation inplace and return None.

Returns: DataFrame or None

DataFrame with NA entries dropped from it or None if `inplace=True`.

pandas.DataFrame.fillna

`DataFrame.fillna(value=None, method=None, axis=None, inplace=False, limit=None, downcast=None)` [\[source\]](#)

Fill NA/NaN values using the specified method.

Parameters: **value** : scalar, dict, Series, or DataFrame

Value to use to fill holes (e.g. 0), alternately a dict/Series/DataFrame of values specifying which value to use for each index (for a Series) or column (for a DataFrame). Values not in the dict/Series/DataFrame will not be filled. This value cannot be a list.

method : {'backfill', 'bfill', 'pad', 'ffill', None}, default None

Method to use for filling holes in reindexed Series pad / ffill: propagate last valid observation forward to next valid backfill / bfill: use next valid observation to fill gap.

axis : {0 or 'index', 1 or 'columns'}

Axis along which to fill missing values.

inplace : bool, default False

If True, fill in-place. Note: this will modify any other views on this object (e.g., a no-copy slice for a column in a DataFrame).

limit : int, default None

If method is specified, this is the maximum number of consecutive NaN values to forward/backward fill. In other words, if there is a gap with more than this number of consecutive NaNs, it will only be partially filled. If method is not specified, this is the maximum number of entries along the entire axis where NaNs will be filled. Must be greater than 0 if not None.

downcast : dict, default is None

A dict of item->dtype of what to downcast if possible, or the string 'infer' which will try to downcast to an appropriate equal type (e.g. float64 to int64 if possible).

Returns: DataFrame or None

Object with missing values filled or None if `inplace=True`.



Missing Data 처리 > Replacing

pandas.DataFrame.replace

`DataFrame.replace(to_replace=None, value=NoDefault.no_default, inplace=False, limit=None, regex=False, method=NoDefault.no_default)` [\[source\]](#)

Replace values given in `to_replace` with `value`.

Values of the DataFrame are replaced with other values dynamically.

This differs from updating with `.loc` or `.iloc`, which require you to specify a location to update with some value.

Parameters: `to_replace` : *str, regex, list, dict, Series, int, float, or None*

How to find the values that will be replaced.

- numeric, str or regex:
 - numeric: numeric values equal to `to_replace` will be replaced with `value`
 - str: string exactly matching `to_replace` will be replaced with `value`
 - regex: regexs matching `to_replace` will be replaced with `value`
- list of str, regex, or numeric:
 - First, if `to_replace` and `value` are both lists, they **must** be the same length.
 - Second, if `regex=True` then all of the strings in **both** lists will be interpreted as regexs otherwise they will match directly. This doesn't matter much for `value` since there are only a few possible substitution regexes you can use.
 - str, regex and numeric rules apply as above.
- dict:
 - Dicts can be used to specify different replacement values for different existing values. For example, `{ 'a': 'b', 'y': 'z' }` replaces the value 'a' with 'b' and 'y' with 'z'. To use a dict in this way the `value` parameter should be `None`.
 - For a DataFrame a dict can specify that different values should be replaced in different columns. For example, `{ 'a': 1, 'b': 'z' }` looks for the value 1 in column 'a' and the value 'z' in column 'b' and replaces these values with whatever is specified in `value`. The `value` parameter should not be `None` in this case. You can treat this as a special case of passing two lists except that you are specifying the column to search in.
 - For a DataFrame nested dictionaries, e.g., `{ 'a': { 'b': np.nan} }`, are read as follows: look in column 'a' for the value 'b' and replace it with NaN. The `value` parameter should be `None` to use a nested dict in this way. You can nest regular expressions as well. Note that column names (the top-level dictionary keys in a nested dictionary) **cannot** be regular expressions.

value : *scalar, dict, list, str, regex, default None*

Value to replace any values matching `to_replace` with. For a DataFrame a dict of values can be used to specify which value to use for each column (columns not in the dict will not be filled). Regular expressions, strings and lists or dicts of such objects are also allowed.

inplace : *bool, default False*

If True, performs operation inplace and returns None.

limit : *int, default None*

Maximum size gap to forward or backward fill.

regex : *bool or same types as to_replace, default False*

Whether to interpret `to_replace` and/or `value` as regular expressions. If this is `True` then `to_replace` must be a string. Alternatively, this could be a regular expression or a list, dict, or array of regular expressions in which case `to_replace` must be `None`.

method : *{ 'pad', 'ffill', 'bfill', None }*

The method to use when for replacement, when `to_replace` is a scalar, list or tuple and `value` is `None`.

Changed in version 0.23.0: Added to DataFrame.

Returns: `DataFrame`

Object after replacement.

Raises: `AssertionError`

- If `regex` is not a `bool` and `to_replace` is not `None`.

`TypeError`

- If `to_replace` is not a scalar, array-like, `dict`, or `None`
- If `to_replace` is a `dict` and `value` is not a `list`, `dict`, `ndarray`, or `Series`
- If `to_replace` is `None` and `regex` is not compilable into a regular expression or is a list, `dict`, `ndarray`, or `Series`.
- When replacing multiple `bool` or `datetime64` objects and the arguments to `to_replace` does not match the type of the value being replaced

`ValueError`

- If a `list` or an `ndarray` is passed to `to_replace` and `value` but they are not the same length.



Missing Data 처리 > Replacing

Scalar `to\_replace` and `value`

```
>>> s = pd.Series([1, 2, 3, 4, 5])
>>> s.replace(1, 5)
0    5
1    2
2    3
3    4
4    5
dtype: int64
```

```
>>> df = pd.DataFrame({'A': [0, 1, 2, 3, 4],
...                    'B': [5, 6, 7, 8, 9],
...                    'C': ['a', 'b', 'c', 'd', 'e']})
>>> df.replace(0, 5)
   A  B  C
0  5  5  a
1  1  6  b
2  2  7  c
3  3  8  d
4  4  9  e
```

List-like `to\_replace`

```
>>> df.replace([0, 1, 2, 3], 4)
   A  B  C
0  4  5  a
1  4  6  b
2  4  7  c
3  4  8  d
4  4  9  e
```

```
>>> df.replace([0, 1, 2, 3], [4, 3, 2, 1])
   A  B  C
0  4  5  a
1  3  6  b
2  2  7  c
3  1  8  d
4  4  9  e
```

```
>>> s.replace([1, 2], method='bfill')
0    3
1    3
2    3
3    4
4    5
dtype: int64
```

dict-like `to\_replace`

```
>>> df.replace({0: 10, 1: 100})
   A  B  C
0  10  5  a
1  100 6  b
2    2  7  c
3    3  8  d
4    4  9  e
```

```
>>> df.replace({'A': 0, 'B': 5}, 100)
   A  B  C
0  100 100 a
1    1    6 b
2    2    7 c
3    3    8 d
4    4    9 e
```

```
>>> df.replace({'A': {0: 100, 4: 400}})
   A  B  C
0  100  5  a
1    1    6 b
2    2    7 c
3    3    8 d
4  400  9  e
```


수치형 -> 범주화 (Discretization / Binning)

pandas.cut

`pandas.cut(x, bins, right=True, labels=None, retbins=False, precision=3, include_lowest=False, duplicates='raise', ordered=True)` [\[source\]](#)

Bin values into discrete intervals.

Use `cut` when you need to segment and sort data values into bins. This function is also useful for going from a continuous variable to a categorical variable. For example, `cut` could convert ages to groups of age ranges. Supports binning into an equal number of bins, or a pre-specified array of bins.

Parameters: **x** : *array-like*

The input array to be binned. Must be 1-dimensional.

bins : *int, sequence of scalars, or IntervalIndex*

The criteria to bin by.

- `int` : Defines the number of equal-width bins in the range of `x`. The range of `x` is extended by .1% on each side to include the minimum and maximum values of `x`.
- `sequence of scalars` : Defines the bin edges allowing for non-uniform width. No extension of the range of `x` is done.
- `IntervalIndex` : Defines the exact bins to be used. Note that `IntervalIndex` for `bins` must be non-overlapping.

right : *bool, default True*

Indicates whether `bins` includes the rightmost edge or not. If `right == True` (the default), then the `bins` `[1, 2, 3, 4]` indicate (1,2], (2,3], (3,4]. This argument is ignored when `bins` is an `IntervalIndex`.

labels : *array or False, default None*

Specifies the labels for the returned bins. Must be the same length as the resulting bins. If `False`, returns only integer indicators of the bins. This affects the type of the output container (see below). This argument is ignored when `bins` is an `IntervalIndex`. If `True`, raises an error. When `ordered=False`, labels must be provided.

retbins : *bool, default False*

Whether to return the bins or not. Useful when `bins` is provided as a scalar.

precision : *int, default 3*

The precision at which to store and display the bins labels.

include_lowest : *bool, default False*

Whether the first interval should be left-inclusive or not.

duplicates : *{default 'raise', 'drop'}, optional*

If bin edges are not unique, raise `ValueError` or drop non-uniques.

ordered : *bool, default True*

Whether the labels are ordered or not. Applies to returned types `Categorical` and `Series` (with `Categorical` dtype). If `True`, the resulting categorical will be ordered. If `False`, the resulting categorical will be unordered (labels must be provided).

Returns: **out** : *Categorical, Series, or ndarray*

An array-like object representing the respective bin for each value of `x`. The type depends on the value of `labels`.

- `None` (default) : returns a `Series` for `Series x` or a `Categorical` for all other inputs. The values stored within are `Interval` dtype.
- `sequence of scalars` : returns a `Series` for `Series x` or a `Categorical` for all other inputs. The values stored within are whatever the type in the sequence is.
- `False` : returns an `ndarray` of integers.

bins : *numpy.ndarray or IntervalIndex.*

The computed or specified bins. Only returned when `retbins=True`. For scalar or sequence `bins`, this is an `ndarray` with the computed bins. If set `duplicates=drop`, `bins` will drop non-unique bin. For an `IntervalIndex bins`, this is equal to `bins`.

Examples

Discretize into three equal-sized bins.

```
>>> pd.cut(np.array([1, 7, 5, 4, 6, 3]), 3)
...
[(0.994, 3.0], (5.0, 7.0], (3.0, 5.0], (3.0, 5.0], (5.0, 7.0], ...
Categories (3, interval[float64, right]): [(0.994, 3.0] < (3.0, 5.0] ...
```

```
>>> pd.cut(np.array([1, 7, 5, 4, 6, 3]), 3, retbins=True)
...
[(0.994, 3.0], (5.0, 7.0], (3.0, 5.0], (3.0, 5.0], (5.0, 7.0], ...
Categories (3, interval[float64, right]): [(0.994, 3.0] < (3.0, 5.0] ...
array([0.994, 3.    , 5.    , 7.    ])
```

Discovers the same bins, but assign them specific labels. Notice that the returned `Categorical`'s categories are `labels` and is ordered.

```
>>> pd.cut(np.array([1, 7, 5, 4, 6, 3]),
...         3, labels=["bad", "medium", "good"])
['bad', 'good', 'medium', 'medium', 'good', 'bad']
Categories (3, object): ['bad' < 'medium' < 'good']
```

`ordered=False` will result in unordered categories when labels are passed. This parameter can be used to allow non-unique labels:

```
>>> pd.cut(np.array([1, 7, 5, 4, 6, 3]), 3,
...         labels=["B", "A", "B"], ordered=False)
['B', 'B', 'A', 'A', 'B', 'B']
Categories (2, object): ['A', 'B']
```

`labels=False` implies you just want the bins back.

```
>>> pd.cut([0, 1, 1, 2], bins=4, labels=False)
array([0, 1, 1, 3])
```




수치형 -> 범주화 (*Discretization / Binning*)

pandas.qcut

`pandas.qcut(x, q, labels=None, retbins=False, precision=3, duplicates='raise')`

Quantile-based discretization function.

[\[source\]](#)

Discretize variable into equal-sized buckets based on rank or based on sample quantiles. For example 1000 values for 10 quantiles would produce a Categorical object indicating quantile membership for each data point.

Parameters: **x** : *1d ndarray or Series*

q : *int or list-like of float*

Number of quantiles. 10 for deciles, 4 for quartiles, etc. Alternately array of quantiles, e.g. [0, .25, .5, .75, 1.] for quartiles.

labels : *array or False, default None*

Used as labels for the resulting bins. Must be of the same length as the resulting bins. If False, return only integer indicators of the bins. If True, raises an error.

retbins : *bool, optional*

Whether to return the (bins, labels) or not. Can be useful if bins is given as a scalar.

precision : *int, optional*

The precision at which to store and display the bins labels.

duplicates : *{default 'raise', 'drop'}, optional*

If bin edges are not unique, raise ValueError or drop non-uniques.

Returns: **out** : *Categorical or Series or array of integers if labels is False*

The return type (Categorical or Series) depends on the input: a Series of type category if input is a Series else Categorical. Bins are represented as categories when categorical data is returned.

bins : *ndarray of floats*

Returned only if *retbins* is True.

Examples

```
>>> pd.qcut(range(5), 4)
...
[(-0.001, 1.0], (-0.001, 1.0], (1.0, 2.0], (2.0, 3.0], (3.0, 4.0]]
Categories (4, interval[float64, right]): [(-0.001, 1.0] < (1.0, 2.0] ...
```

```
>>> pd.qcut(range(5), 3, labels=["good", "medium", "bad"])
...
[good, good, medium, bad, bad]
Categories (3, object): [good < medium < bad]
```

```
>>> pd.qcut(range(5), 4, labels=False)
array([0, 0, 1, 2, 3])
```



Outlier Detection & Filtering > 정규분포 기준

For Normal distributions: Use empirical relations of Normal distribution.

The data points which fall below $mean - 3 \times (sigma)$ or above $mean + 3 \times (sigma)$ are outliers.

where mean and sigma are the average value and standard deviation of a particular column.

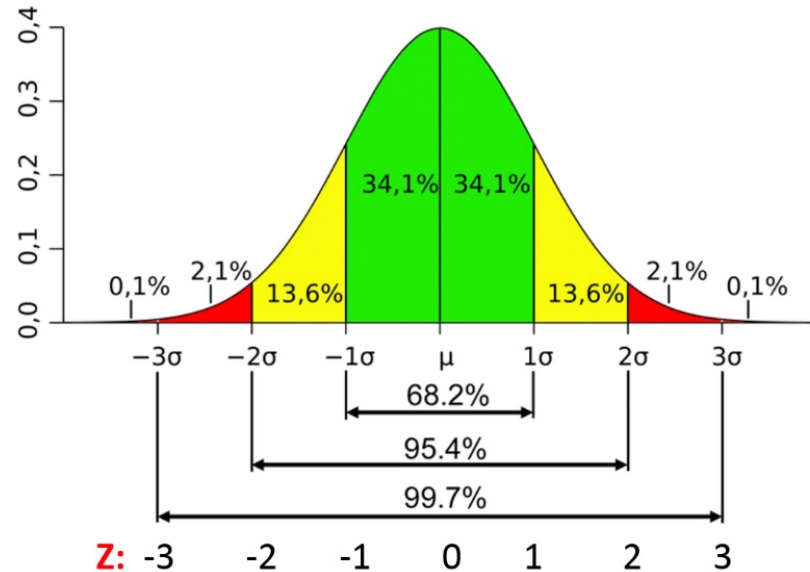


Fig. Characteristics of a Normal Distribution

```
print("Highest allowed",df['cgpa'].mean() + 3*df['cgpa'].std())
print("Lowest allowed",df['cgpa'].mean() - 3*df['cgpa'].std())
```

Output:

```
Highest allowed 8.808933625397177
Lowest allowed 5.113546374602842
```

Step-5: Finding the Outliers

```
df[(df['cgpa'] > 8.80) | (df['cgpa'] < 5.11)]
```

Step-6: Trimming of Outliers

```
new_df = df[(df['cgpa'] < 8.80) & (df['cgpa'] > 5.11)]
new_df
```

Step-7: Capping on Outliers

```
upper_limit = df['cgpa'].mean() + 3*df['cgpa'].std()
lower_limit = df['cgpa'].mean() - 3*df['cgpa'].std()
```

Step-8: Now, apply the Capping

```
df['cgpa'] = np.where(
    df['cgpa'] > upper_limit,
    upper_limit,
    np.where(
        df['cgpa'] < lower_limit,
        lower_limit,
        df['cgpa']
    )
)
```

<https://www.analyticsvidhya.com/blog/2021/05/feature-engineering-how-to-detect-and-remove-outliers-with-python-code/>



Outlier Detection & Filtering > IQR 기준

👉 IQR based filtering :

Used when our data distribution is skewed.

Step-5: Finding the IQR

```
percentile25 = df['placement_exam_marks'].quantile(0.25)
percentile75 = df['placement_exam_marks'].quantile(0.75)
```

Step-6: Finding upper and lower limit

```
upper_limit = percentile75 + 1.5 * iqr
lower_limit = percentile25 - 1.5 * iqr
```

Step-7: Finding Outliers

```
df[df['placement_exam_marks'] > upper_limit]
df[df['placement_exam_marks'] < lower_limit]
```

Step-8: Trimming

```
new_df = df[df['placement_exam_marks'] < upper_limit]
new_df.shape
```

<https://www.analyticsvidhya.com/blog/2021/05/feature-engineering-how-to-detect-and-remove-outliers-with-python-code/>





Indexing / Slicing / Masking / Selection

indexing: getting a specific element

grades = [88, 72, 93, 94]

0 1 2 3

```
>>> grades[2]
93
```

slicing: selecting a set of elements

grades = [88, 72, 93, 94]

0 1 2 3 4

```
>>> grades[1:3]
[72, 93]
```

Boolean 인덱싱하기

- 기존 DataFrame에서, "year"칼럼 중 2014보다 큰 값만 인덱싱하고 싶다고 가정해보자.
df["year"] > 2014 를 입력하면, Series형태로 마스크가 얻어진다.

```
In [70]: df["year"] > 2014
Out [70]: Order
one      False
two      True
threes   True
four     True
five     True
six      False
seven    True
Name: year, dtype: bool
```

(마스크는 전체데이터중 기준이 되는 열 과 그 가운데 특정성분을 비교한 조건문이었다.
for문으로 선택된 행을 뽑아내고 싶을 땐, 중복없는 전체기준열을 따로 담아서, 조건문에 하나씩 들어가도록 사용했었다)

- 이러한 마스크를 인덱스 자리에 넣은 .loc함수를 이용하면 원하는 행을 선택할 수 있다. (열은 전체를 선택한다.)
df.loc[df["year"] > 2014 , :]

```
In [71]: df.loc[ df["year"] > 2014 , : ]
Out [71]:
```

	Info	year	names	points	penalty
Order					
two	2015.0	Kilho	1.7	-2	
threes	2016.0	Kilho	3.6	-3	
four	2015.0	Charles	2.4	-4	
five	2016.0	Charles	2.9	-5	
seven	2017.0	Cho	4.0	-7	

<https://datacarpentry.org/python-ecology-lesson/03-index-slice-subset/>
<https://nittaku.tistory.com/111>



정수 *indexing*

.iloc 메서드(정수 위치)를 이용한 인덱싱

본 포스팅에서는 .iloc 메서드에 대해 좀 더 상세히 알아볼 것이며, 이 메서드를 이용한 인덱싱에 대해 살펴볼 것이다.

.iloc 메서드에 대한 설명

pandas는 순수하게 정수 기반 인덱싱을 위한 일련의 메서드들을 제공한다.

.iloc 메서드는 주로 정수 위치(integer position) 기반으로 사용된다.

○ 여기서 정수 위치 인덱싱은 0에서 축의 길이-1에 해당된다.

.iloc 메서드의 특징

.iloc 메서드의 특징으로는 다음과 같다.

- python 인덱싱과 동일하게 0부터 시작한다. (0-based 인덱싱)
- 파이썬과 Numpy 슬라이싱에 거의 가깝게 따른다.
 - ▷ 슬라이싱을 사용할때 시작지점은 포함되어 있는 반면 마지막 입력지점은 제외된다.(python 슬라이싱과 유사)
- 불린 배열(boolean array)를 사용하는 것이 가능하다.
- 만약 입력된 인덱스가 범위를 넘어서는 경우에는 IndexError를 발생시킨다.
 - ▷ 슬라이싱을 할때에는 예외적으로 인덱스 범위를 넘어서도 된다.
- 비정수 인덱싱을 사용할 경우 IndexError를 일으킨다.

.iloc 메서드의 사용법

.iloc 메서드는 .loc와 마찬가지로 .iloc 뒤에 대괄호를 붙여 사용한다.

구문 형식은 다음과 같다.

.loc 메서드의 기본적인 구문 형식)

Series에서의 사용

① Series.iloc[index_int] # Series에서의 요소 선택

DataFrame에서의 사용

② DataFrame.iloc[index_int] # 행의 선택

③ DataFrame.iloc[index_int, column_int] # 하나의 요소 선택

index_int은 행을 선택하기 위한 인덱스의 위치 정수를 의미한다.

column_int은 행 선택 이후 해당 행에서 요소를 선택하기 위한 열의 위치 정수를 의미한다.

각 변수 위치에 슬라이싱을 입력해도 유효하다.

- ①은 Series에서 데이터를 선택하는 형식이며 하나의 요소를 반환한다.
- ②은 DataFrame에서 데이터를 선택하는 형식이며 행에 해당하는 Series를 반환한다.
- ③은 DataFrame에서 데이터를 선택하는 형식이며, 열의 레이블을 입력함으로써 하나의 요소를 선택하여 반환한다.





데이터프레임의 산술연산

- 1. Column 과 Column 간 연산 (+, -, *, /, %)
- 2. Column 과 숫자 간 연산 (+, -, *, /, %)
- 3. 복합 연산
- 4. mean(), sum() 을 axis 기준으로 연산
- 5. NaN 값이 존재할 경우 연산
- 6. DataFrame 과 DataFrame 간 연산
 - 6-1. 문자열이 포함된 Series / DataFrame의 연산은 불가하다
 - 6-2. 두 DataFrame의 column 이름은 같으나 column 순서만 바뀌어 있는 경우
 - 6-3. 행의 갯수가 다른 경우

<https://hyemin-kim.github.io/2020/06/20/S-Python-Pandas-Pre6/>





데이터 정렬 (*sort_index()* / *sort_values()*)

.sort_index()를 사용하여 데이터 정렬하기

In [1]:

```
# 판다스 라이브러리 임포트
import pandas as pd
# plotly express에 내장되어있는 gapminder 데이터프레임을 사용하겠습니다.
import plotly.express as px
df = px.data.gapminder()

# 데이터 df를 인덱스를 기준으로 정렬합니다.
# 기본 정렬 방식은 오름차순(ascending)입니다.
# 내림차순으로 정렬하고 싶다면 ascending 옵션을 False로 설정하면 됩니다.
df_sorted_by_index = df.sort_index(ascending=False)

# 정렬 결과를 출력합니다.
df_sorted_by_index.head()
```

Out[1]:

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	iso_num
1703	Zimbabwe	Africa	2007	43.487	12311143	469.709298	ZWE	716
1702	Zimbabwe	Africa	2002	39.989	11926563	672.038623	ZWE	716
1701	Zimbabwe	Africa	1997	46.809	11404948	792.449960	ZWE	716
1700	Zimbabwe	Africa	1992	60.377	10704340	693.420786	ZWE	716
1699	Zimbabwe	Africa	1987	62.351	9216418	706.157306	ZWE	716

.sort_values()를 사용하여 데이터 정렬하기

sort_values()의 경우에는 어떤 컬럼의 값을 기준으로 정렬해야 하는지 명시적으로 설정해야하기 때문에, by 옵션에 정렬 기준이 되는 컬럼명을 집어넣습니다.

In [2]:

```
# 데이터 df를 year 컬럼의 값을 기준으로 정렬합니다.
# 기본 정렬 방식은 오름차순(ascending)입니다.
# 내림차순으로 정렬하고 싶다면 ascending 옵션을 False로 설정하면 됩니다.
df_sorted_by_values = df.sort_values(by='year', ascending=False)

# 정렬 결과를 출력합니다.
df_sorted_by_values.head()
```

Out[2]:

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	iso_num
1703	Zimbabwe	Africa	2007	43.487	12311143	469.709298	ZWE	716
491	Equatorial Guinea	Africa	2007	51.579	551201	12154.089750	GNQ	226
515	Ethiopia	Africa	2007	52.947	76511887	690.805576	ETH	231
527	Finland	Europe	2007	79.313	5238460	33207.084400	FIN	246
539	France	Europe	2007	80.657	61083916	30470.016700	FRA	250

<https://hogni.tistory.com/6>





정렬 및 순위

그림 4.12.1 데이터프레임 객체

그림 4.12.1와 같은 데이터프레임 객체가 있을 때 현재가를 기준으로 데이터프레임 객체를 정렬해봅시다. 데이터프레임 객체는 여러 개의 칼럼으로 구성되기 때문에 `sort_values(by='칼럼 이름')` 과 같은 형태로 메서드를 사용하면 됩니다.

```
# 04/35.py
01: from pandas import DataFrame
02:
03: data = [
04:     ["037730", "3R", 1510],
05:     ["036360", "3SOFT", 1790],
06:     ["005760", "ACTS", 1185],
07: ]
08:
09: columns = ["종목코드", "종목명", "현재가"]
10: df = DataFrame(data=data, columns=columns)
11: df = df.set_index('종목코드')
12:
13: # 현재가를 기준으로 정렬
14: df2 = df.sort_values(by='현재가')
15: print(df2)
```

라인 10: 데이터프레임 객체를 생성합니다.

라인 11: '종목코드' 칼럼을 데이터프레임의 인덱스로 설정합니다.

라인 14: '현재가' 칼럼을 기준으로(오름차순) 데이터프레임을 정렬하고 정렬된 데이터프레임 객체를 df2라는 변수가 바인딩합니다.

`sort_values(by='현재가')` 를 통해 정렬된 df2에는 그림 4.12.2과 같이 데이터가 저장됩니다. 만일 현재가를 기준으로 내림차순 정렬하려면 `sort_values(by='현재가', ascending=False)` 를 사용합니다. 그 결과 그림 4.12.3의 데이터프레임을 얻을 수 있습니다.

종목명	현재가
종목코드	
005760	ACTS 1185
037730	3R 1510
036360	3SOFT 1790

그림 4.12.2 데이터프레임 오름차순 정렬

순위

이번에는 현재가가 낮은 종목부터 순위를 매겨 보겠습니다. 순위는 데이터프레임에서 칼럼을 선택한 후 `rank()` 메서드를 호출하면 됩니다. 그림 4.12.4와 같이 순위를 매긴 후에 해당 칼럼을 기존의 데이터프레임에 추가해주는 것이 데이터를 관리하기가 편리합니다.

종목명	현재가	순위
종목코드		
037730	3R 1510	2.0
036360	3SOFT 1790	3.0
005760	ACTS 1185	1.0

그림 4.12.4 순위를 매기고 칼럼으로 추가

```
# 04/36.py
# 코드 생략
13: # 현재가를 기준으로 순위
14: df['순위'] = df['현재가'].rank()
15: print(df)
```

라인 14: '현재가' 칼럼을 기준으로 순위가 매겨진 시리즈 객체를 만든 후 이를 '순위'라는 칼럼으로 데이터프레임 객체에 추가합니다.

순위를 기준으로 정렬하려면 앞서 배운 `sort_values(by='순위')` 메서드를 호출하면 되겠지요? 다음과 같이 코드를 한 줄 추가하면 순위가 추가된 데이터프레임 객체를 '순위'를 기준으로 정렬할 수 있습니다.

종목명	현재가	순위
종목코드		
005760	ACTS 1185	1.0
037730	3R 1510	2.0
036360	3SOFT 1790	3.0



함수 적용 및 매핑(Mapping)

함수 매핑(mapping)

함수 매핑이란 시리즈나 데이터프레임의 개별 원소를 특정 함수에 일대일 대응시키는 과정을 의미한다. 사용자가 직접 만든 함수(lambda 함수 포함)를 적용할 수 있기 때문에 for문이나 기본 함수로 처리하는 것보다 훨씬 효율적인 장점이 있다.

1. 개별 원소에 함수 매핑

1-1. Series 객체.apply() 함수

이 함수는 시리즈 객체의 개별 원소에 함수를 적용할 때 사용한다.

1-2. DataFrame 객체.applymap() 함수

이 함수는 데이터프레임의 개별 원소에 함수를 적용할 때 사용한다.

2. 시리즈 객체에 함수 매핑

2-1. DataFrame 객체.apply(매핑 함수, axis =) 함수

데이터프레임의 열 또는 행에 대해 함수를 적용시키고 싶다면, apply 함수를 적용하면 된다.

3. 데이터프레임 객체에 함수 매핑

3-1. DataFrame 객체.pipe() 함수

`pipe()` 함수는 데이터프레임 객체에 바로 적용시키는 함수로, 입력인자를 들어가는 매핑 함수의 반환 형태를 그대로 따라서 값을 반환해준다.

https://yganalyst.github.io/data_handling/Pd_9/





범주형 -> 수치화 (Dummy Variable)

pandas.get_dummies

`pandas.get_dummies(data, prefix=None, prefix_sep='_', dummy_na=False, columns=None, sparse=False, drop_first=False, dtype=None)` [\[source\]](#)

Convert categorical variable into dummy/indicator variables.

Parameters: **data** : array-like, Series, or DataFrame

Data of which to get dummy indicators.

prefix : str, list of str, or dict of str, default None

String to append DataFrame column names. Pass a list with length equal to the number of columns when calling get_dummies on a DataFrame. Alternatively, *prefix* can be a dictionary mapping column names to prefixes.

prefix_sep : str, default '_'

If appending prefix, separator/delimiter to use. Or pass a list or dictionary as with *prefix*.

dummy_na : bool, default False

Add a column to indicate NaNs, if False NaNs are ignored.

columns : list-like, default None

Column names in the DataFrame to be encoded. If *columns* is None then all the columns with *object* or *category* dtype will be converted.

sparse : bool, default False

Whether the dummy-encoded columns should be backed by a *SparseArray* (True) or a regular NumPy array (False).

drop_first : bool, default False

Whether to get k-1 dummies out of k categorical levels by removing the first level.

dtype : dtype, default np.uint8

Data type for new columns. Only a single dtype is allowed.

Returns: **DataFrame**

Dummy-coded data.

Examples

```
>>> s = pd.Series(list('abca'))
```

```
>>> pd.get_dummies(s)
   a  b  c
0  1  0  0
1  0  1  0
2  0  0  1
3  1  0  0
```

```
>>> s1 = ['a', 'b', np.nan]
```

```
>>> pd.get_dummies(s1)
   a  b
0  1  0
1  0  1
2  0  0
```

```
>>> pd.get_dummies(s1, dummy_na=True)
   a  b  NaN
0  1  0    0
1  0  1    0
2  0  0    1
```

```
>>> df = pd.DataFrame({'A': ['a', 'b', 'a'], 'B': ['b', 'a', 'c'],
...                    'C': [1, 2, 3]})
```

```
>>> pd.get_dummies(df, prefix=['col1', 'col2'])
   C  col1_a  col1_b  col2_a  col2_b  col2_c
0  1         1         0         0         1         0
1  2         0         1         1         0         0
2  3         1         0         0         0         1
```



문자열 다루기 (*series.str*)

pandas.Series.str

Series.**str**()

[\[source\]](#)

Vectorized string functions for Series and Index.

NAs stay NA unless handled otherwise by a particular method. Patterned after Python's string methods, with some inspiration from R's stringr package.

Examples

```
>>> s = pd.Series(["A_Str_Series"])
>>> s
0    A_Str_Series
dtype: object
```

```
>>> s.str.split("_")
0    [A, Str, Series]
dtype: object
```

```
>>> s.str.replace("_", "")
0    AStrSeries
dtype: object
```

1. 인덱싱 **.str[]**
2. 분할 **.str.split()**
3. 시작글자 인식 **.str.startswith()**
4. 끝글자 인식 **.str.endswith()**
5. 포함글자 인식 **.str.contains()**
6. 문자 위치찾기
7. 문자 대체 **.str.replace()**
8. 원하는 문자열 추출 **str.extract()**
9. 문자열 패딩
10. 공백제거 **strip()**
11. 대소문자 변경

https://yganalyst.github.io/data_handling/memo_9/



계층적 인덱싱 (Multilevel Indexing)

계층적 인덱싱

계층적인 정보를 인덱스로 사용하고자 할 경우, Series 혹은 DataFrame에서 '계층적 인덱싱(hierarchical indexing)'을 활용할 수 있습니다.

Series의 계층적 인덱싱

```
s = pd.Series(np.random.randn(10),
              index=[["a", "a", "a", "b", "b", "b", "c", "c", "d", "d"],
                    [1, 2, 3, 1, 2, 3, 1, 2, 2, 3]])
```

위와 같이 Series의 인덱스에 2차원 리스트 혹은 2차원 numpy array 등을 입력하게 되면, 두 층을 가지는 인덱스가 생성됩니다. s 인덱스의 맨 첫 번째 층에 'a'부터 'd'까지, 이들 각각에 대하여 정수 인덱스가 두 번째 층에 부여되어 있습니다.

계층적 인덱싱을 수행할 때의 원칙은, 항상 가장 상위 계층에 해당하는 맨 바깥쪽 인덱스부터 차례대로 인덱싱하는 것입니다. 예를 들어 `s["b"]` 와 같이 첫 번째 층의 'b' 인덱스를 명시하면, 해당 인덱스의 하위 층에 있는 1부터 3까지의 인덱스와 값을 Series 형태로 얻습니다. `s["b": "c"]` 와 같이 첫 번째 층에 범위 인덱싱을 적용할 수도 있습니다.

`s[("b", 3)]` 을 실행하면 'b' 인덱스 하위 층의 3 인덱스에 해당하는 값을 얻습니다. 또 는, `s[:, 2]` 를 실행하면 모든 첫 번째 층의 인덱스에 대하여 하위 계층의 2 인덱스의 값 들만으로 구성된 Series를 얻을 수 있습니다.

DataFrame의 계층적 인덱싱

```
df = pd.DataFrame(np.arange(12).reshape((4, 3)),
                  index=[["a", "a", "b", "b"],
                        [1, 2, 1, 2]],
                  columns=["Seoul", "Seoul", "Busan"],
                  ["Green", "Red", "Green"]])
```

위와 같이 DataFrame의 인덱스와 컬럼에 2차원 리스트 혹은 2차원 numpy array 등을 입력하게 되면, 두 층을 가지는 인덱스 혹은 컬럼이 생성됩니다. 복수 개 층의 인덱스와 컬럼에 이름을 붙일 때는, 다음과 같이 맨 바깥쪽의 인덱스와 컬럼부터 순서대로 붙이면 됩니다.

```
dfd.index.names = ["key1", "key2"]
df.columns.names = ["city", "color"]
```

`df["Seoul"]` 을 실행하면 첫 번째 층의 컬럼이 "Seoul"인 데이터를 DataFrame의 형태로 얻습니다. 기존의 두 번째 층의 컬럼 중 "Seoul"의 하위에 있던 것들만이 새로운 DataFrame의 형태로 얻어집니다. `df[("Seoul", "Green")]` 을 실행하면, "Seoul" 컬럼 하위 층의 "Green" 컬럼에 해당하는 열을 Series 형태로 얻습니다. 해당 Series의 인덱스는 여전히 계층적인 형태임이 확인 가능합니다.

`df.loc["a"]` 을 실행하면 첫 번째 층 인덱스가 "a"인 데이터를 DataFrame의 형태로 얻으며, `df.loc[("a", 1)]` 을 실행하면 "a" 인덱스 하위 층의 1 인덱스에 해당하는 행을 Series 형태로 얻습니다. 이 때 해당 Series의 인덱스는 기존에 있던 계층적 컬럼이 들어왔습니다.

https://freelife1191.github.io/dev/2018/05/07/dev-data_analysis-22.python_data_analysis/

AI 데이터 합치기 & 조인 (Concat & Merge & Join)

1. 데이터 프레임 붙이기 : `pd.concat()`

1-1. `pd.concat()`의 옵션

1-2. 시리즈를 데이터프레임에 붙이기

1-3. 시리즈 끼리 붙이기

2. 데이터프레임 병합 : `pd.merge()`

2-1. `merge()`함수의 옵션

3. 데이터프레임 결합 : `join()`

1. 데이터 프레임 붙이기 : `pd.concat()`

`pd.concat()` 함수는 데이터프레임을 말그대로 물리적으로 이어 붙여주는 함수로,
`pd.concat(데이터프레임리스트)`로 사용한다.

2. 데이터프레임 병합 : `pd.merge()`

`merge()` 함수는 두 데이터프레임을 각 데이터에 존재하는 고유값(key)을 기준으로 병합할때 사용한다.
`pd.merge(df_left, df_right, how='inner', on=None)` 이 default이다.

3. 데이터프레임 결합 : `join()`

`join`함수는 `merge()`함수를 기반으로 만들어졌기 때문에 기본 작동방식이 비슷하다. 하지만 `join()`은 행 인덱스를 기준으로 결합한다는 점에서 차이가 있다. 그래도 이 함수도 `on=keys` 옵션이 존재한다.

`Dataframe1.join(Dataframe2, how='left')` 이 default값이다.

https://yganalyst.github.io/data_handling/Pd_12/





시계열 데이터

1. 시계열 데이터

2. 자료형의 시계열 객체 변환 : `to_datetime()` , `to_period()`

3. 시계열 데이터 만들기 : `date_range()` , `period_range()`

3-1. Timestamp 배열

3-2. Period 배열

4. 시계열데이터 활용

4-1. 날짜 데이터 분리 : `dt.year`, `dt.month`, `dt.day`

4-2. 날짜 인덱스 활용

4-3. 오늘과의 날짜 차이 열 만들기

4-4. 오늘날짜 : `datetime` 모듈

https://yganalyst.github.io/data_handling/Pd_8/





기초 통계량 (describe)

pandas.DataFrame.describe

`DataFrame.describe(percentiles=None, include=None, exclude=None, datetime_is_numeric=False)`

[\[source\]](#)

Generate descriptive statistics.

Descriptive statistics include those that summarize the central tendency, dispersion and shape of a dataset's distribution, excluding `NaN` values.

Analyzes both numeric and object series, as well as `DataFrame` column sets of mixed data types. The output will vary depending on what is provided. Refer to the notes below for more detail.

Parameters: `percentiles` : list-like of numbers, optional

The percentiles to include in the output. All should fall between 0 and 1. The default is `[.25, .5, .75]`, which returns the 25th, 50th, and 75th percentiles.

include : 'all', list-like of dtypes or None (default), optional

A white list of data types to include in the result. Ignored for `Series`. Here are the options:

- 'all' : All columns of the input will be included in the output.
- A list-like of dtypes : Limits the results to the provided data types. To limit the result to numeric types submit `numpy.number`. To limit it instead to object columns submit the `numpy.object` data type. Strings can also be used in the style of `select_dtypes` (e.g. `df.describe(include=['O'])`). To select pandas categorical columns, use `'category'`
- None (default) : The result will include all numeric columns.

exclude : list-like of dtypes or None (default), optional,

A black list of data types to omit from the result. Ignored for `Series`. Here are the options:

- A list-like of dtypes : Excludes the provided data types from the result. To exclude numeric types submit `numpy.number`. To exclude object columns submit the data type `numpy.object`. Strings can also be used in the style of `select_dtypes` (e.g. `df.describe(exclude=['O'])`). To exclude pandas categorical columns, use `'category'`
- None (default) : The result will exclude nothing.

datetime_is_numeric : bool, default False

Whether to treat datetime dtypes as numeric. This affects statistics calculated for the column. For `DataFrame` input, this also controls whether datetime columns are included by default.

📌 New in version 1.1.0.

Examples

Describing a numeric `Series`.

```
>>> s = pd.Series([1, 2, 3])
>>> s.describe()
count    3.0
mean     2.0
std      1.0
min      1.0
25%     1.5
50%     2.0
75%     2.5
max      3.0
dtype: float64
```

Describing a categorical `Series`.

```
>>> s = pd.Series(['a', 'a', 'b', 'c'])
>>> s.describe()
count    4
unique    3
top      a
freq     2
dtype: object
```

Describing a timestamp `Series`.

```
>>> s = pd.Series([
...     np.datetime64("2000-01-01"),
...     np.datetime64("2010-01-01"),
...     np.datetime64("2010-01-01")
... ])
>>> s.describe(datetime_is_numeric=True)
count    3
mean    2006-09-01 08:00:00
min     2000-01-01 00:00:00
25%    2004-12-31 12:00:00
50%    2010-01-01 00:00:00
75%    2010-01-01 00:00:00
max     2010-01-01 00:00:00
dtype: object
```

Describing a `DataFrame`. By default only numeric fields are returned.

```
>>> df = pd.DataFrame({'categorical': pd.Categorical(['d', 'e', 'f']),
...                   'numeric': [1, 2, 3],
...                   'object': ['a', 'b', 'c']}
...                   )
>>> df.describe()
numeric
count    3.0
mean     2.0
std      1.0
min      1.0
25%     1.5
50%     2.0
75%     2.5
max      3.0
```

Describing all columns of a `DataFrame` regardless of data type.

```
>>> df.describe(include='all')
categorical  numeric  object
count        3      3.0      3
unique        3     NaN      3
top          f     NaN      a
freq          1     NaN      1
mean         NaN     2.0     NaN
std          NaN     1.0     NaN
min          NaN     1.0     NaN
25%          NaN     1.5     NaN
50%          NaN     2.0     NaN
75%          NaN     2.5     NaN
max          NaN     3.0     NaN
```

Returns: `Series` or `DataFrame`

Summary statistics of the Series or Dataframe provided.



기초 통계량 (상관계수 / 공분산)

pandas.DataFrame.corr

`DataFrame.corr(method='pearson', min_periods=1)`

[\[source\]](#)

Compute pairwise correlation of columns, excluding NA/null values.

Parameters: **method** : {'pearson', 'kendall', 'spearman'} or callable

Method of correlation:

- pearson : standard correlation coefficient
- kendall : Kendall Tau correlation coefficient
- spearman : Spearman rank correlation
- callable: callable with input two 1d ndarrays

and returning a float. Note that the returned matrix from corr will have 1 along the diagonals and will be symmetric regardless of the callable's behavior.

min_periods : int, optional

Minimum number of observations required per pair of columns to have a valid result. Currently only available for Pearson and Spearman correlation.

Returns: **DataFrame**

Correlation matrix.

Examples

```
>>> def histogram_intersection(a, b):
...     v = np.minimum(a, b).sum().round(decimals=1)
...     return v
>>> df = pd.DataFrame([(0.2, 0.3), (0.0, 0.6), (0.6, 0.0), (0.2, 0.1)],
...                   columns=['dogs', 'cats'])
>>> df.corr(method=histogram_intersection)
   dogs  cats
dogs  1.0  0.3
cats  0.3  1.0
```




기초 통계량 (Unique Value / Unique Count / Value Counts)

pandas.unique

pandas.unique(values)

Return unique values based on a hash table.

Uniques are returned in order of appearance. This does NOT sort.

Significantly faster than numpy.unique for long enough sequences. Includes NA values.

Parameters: values : 1d array-like

Returns: numpy.ndarray or ExtensionArray

The return can be:

- Index : when the input is an Index
- Categorical : when the input is a Categorical dtype
- ndarray : when the input is a Series/ndarray

Return numpy.ndarray or ExtensionArray.

Examples

```
>>> pd.unique(pd.Series([2, 1, 3, 3]))
array([2, 1, 3])
```

```
>>> pd.unique(pd.Series([2] + [1] * 5))
array([2, 1])
```

```
>>> pd.unique(pd.Series([pd.Timestamp("20160101"), pd.Timestamp("20160101")]))
array(['2016-01-01T00:00:00.000000000', dtype='datetime64[ns]'])
```

pandas.DataFrame.nunique

DataFrame.nunique(axis=0, dropna=True)

Count number of distinct elements in specified axis.

Return Series with number of distinct elements. Can ignore NaN values.

Parameters: axis : {0 or 'index', 1 or 'columns'}, default 0

The axis to use. 0 or 'index' for row-wise, 1 or 'columns' for column-wise.

dropna : bool, default True

Don't include NaN in the counts.

Returns: Series

Examples

```
>>> df = pd.DataFrame({'A': [4, 5, 6], 'B': [4, 1, 1]})
>>> df.nunique()
A    3
B    2
dtype: int64
```

```
>>> df.nunique(axis=1)
0    1
1    2
2    2
dtype: int64
```



기초 통계량 (Unique Value / Unique Count / Value Counts)

pandas.Series.value_counts

`Series.value_counts(normalize=False, sort=True, ascending=False, bins=None, dropna=True)` [\[source\]](#)

Return a Series containing counts of unique values.

The resulting object will be in descending order so that the first element is the most frequently-occurring element. Excludes NA values by default.

Parameters: **normalize** : *bool, default False*

If True then the object returned will contain the relative frequencies of the unique values.

sort : *bool, default True*

Sort by frequencies.

ascending : *bool, default False*

Sort in ascending order.

bins : *int, optional*

Rather than count values, group them into half-open bins, a convenience for `pd.cut`, only works with numeric data.

dropna : *bool, default True*

Don't include counts of NaN.

Returns: **Series**

Examples

```
>>> index = pd.Index([3, 1, 2, 3, 4, np.nan])
>>> index.value_counts()
3.0    2
1.0    1
2.0    1
4.0    1
dtype: int64
```

With *normalize* set to *True*, returns the relative frequency by dividing all values by the sum of values.

```
>>> s = pd.Series([3, 1, 2, 3, 4, np.nan])
>>> s.value_counts(normalize=True)
3.0    0.4
1.0    0.2
2.0    0.2
4.0    0.2
dtype: float64
```

bins

Bins can be useful for going from a continuous variable to a categorical variable; instead of counting unique apparitions of values, divide the index in the specified number of half-open bins.

```
>>> s.value_counts(bins=3)
(0.996, 2.0]    2
(2.0, 3.0]     2
(3.0, 4.0]     1
dtype: int64
```

dropna

With *dropna* set to *False* we can also see NaN index values.

```
>>> s.value_counts(dropna=False)
3.0    2
1.0    1
2.0    1
4.0    1
NaN    1
dtype: int64
```



Aggregation & Grouping

pandas.DataFrame.aggregate

`DataFrame.aggregate(func=None, axis=0, *args, **kwargs)`

[\[source\]](#)

Aggregate using one or more operations over the specified axis.

Parameters: **func** : function, str, list or dict

Function to use for aggregating the data. If a function, must either work when passed a DataFrame or when passed to DataFrame.apply.

Accepted combinations are:

- function
- string function name
- list of functions and/or function names, e.g. `[np.sum, 'mean']`
- dict of axis labels -> functions, function names or list of such.

axis : {0 or 'index', 1 or 'columns'}, default 0

If 0 or 'index': apply function to each column. If 1 or 'columns': apply function to each row.

***args**

Positional arguments to pass to *func*.

****kwargs**

Keyword arguments to pass to *func*.

Returns: **scalar, Series or DataFrame**

The return can be:

- scalar : when Series.agg is called with single function
- Series : when DataFrame.agg is called with a single function
- DataFrame : when DataFrame.agg is called with several functions

Return scalar, Series or DataFrame.

The aggregation operations are always performed over an axis, either the

index (default) or the column axis. This behavior is different from

numpy aggregation functions (*mean*, *median*, *prod*, *sum*, *std*,

var), where the default is to compute the aggregation of the flattened

array, e.g., `numpy.mean(arr_2d)` as opposed to

`numpy.mean(arr_2d, axis=0)`.

agg is an alias for *aggregate*. Use the alias.

Examples

```
>>> df = pd.DataFrame([[1, 2, 3],
...                    [4, 5, 6],
...                    [7, 8, 9],
...                    [np.nan, np.nan, np.nan]],
...                   columns=['A', 'B', 'C'])
```

Aggregate these functions over the rows.

```
>>> df.agg(['sum', 'min'])
      A      B      C
sum  12.0  15.0  18.0
min   1.0   2.0   3.0
```

Different aggregations per column.

```
>>> df.agg({'A' : ['sum', 'min'], 'B' : ['min', 'max']})
      A      B
sum  12.0  NaN
min   1.0   2.0
max   NaN   8.0
```

Aggregate different functions over the columns and rename the index of the resulting DataFrame.

```
>>> df.agg(x=('A', max), y=('B', min), z=('C', np.mean))
      A      B      C
x  7.0  NaN  NaN
y  NaN  2.0  NaN
z  NaN  NaN  6.0
```

Aggregate over the columns.

```
>>> df.agg("mean", axis="columns")
0    2.0
1    5.0
2    8.0
3    NaN
dtype: float64
```



Aggregation & Grouping

pandas.DataFrame.groupby

DataFrame.groupby(by=None, axis=0, level=None, as_index=True, sort=True, group_keys=True, squeeze=None, observed=False, dropna=True)

Group DataFrame using a mapper or by a Series of columns.

[\[source\]](#)

A groupby operation involves some combination of splitting the object, applying a function, and combining the results. This can be used to group large amounts of data and compute operations on these groups.

Parameters: **by** : mapping, function, label, or list of labels

Used to determine the groups for the groupby. If **by** is a function, it's called on each value of the object's index. If a dict or Series is passed, the Series or dict VALUES will be used to determine the groups (the Series' values are first aligned; see `.align()` method). If a list or ndarray of length equal to the selected axis is passed (see the [groupby user guide](#)), the values are used as-is to determine the groups. A label or list of labels may be passed to group by the columns in `self`. Notice that a tuple is interpreted as a (single) key.

axis : {0 or 'index', 1 or 'columns'}, default 0

Split along rows (0) or columns (1).

level : int, level name, or sequence of such, default None

If the axis is a MultiIndex (hierarchical), group by a particular level or levels.

as_index : bool, default True

For aggregated output, return object with group labels as the index. Only relevant for DataFrame input. `as_index=False` is effectively "SQL-style" grouped output.

sort : bool, default True

Sort group keys. Get better performance by turning this off. Note this does not influence the order of observations within each group. Groupby preserves the order of rows within each group.

group_keys : bool, default True

When calling apply, add group keys to index to identify pieces.

squeeze : bool, default False

Reduce the dimensionality of the return type if possible, otherwise return a consistent type.

observed : bool, default False

This only applies if any of the groupers are Categoricals. If True: only show observed values for categorical groupers. If False: show all values for categorical groupers.

dropna : bool, default True

If True, and if group keys contain NA values, NA values together with row/column will be dropped. If False, NA values will also be treated as the key in groups.

! New in version 1.1.0.

Returns:

DataFrameGroupBy

Returns a groupby object that contains information about the groups.

Examples

```
>>> df = pd.DataFrame({'Animal': ['Falcon', 'Falcon',
...                               'Parrot', 'Parrot'],
...                    'Max Speed': [380., 370., 24., 26.]})
>>> df
   Animal  Max Speed
0  Falcon      380.0
1  Falcon      370.0
2  Parrot       24.0
3  Parrot       26.0
>>> df.groupby(['Animal']).mean()
   Animal  Max Speed
Falcon      375.0
Parrot       25.0
```



Pivot Table

pandas.pivot_table

```
pandas.pivot_table(data, values=None, index=None, columns=None, aggfunc='mean',
fill_value=None, margins=False, dropna=True, margins_name='All', observed=False,
sort=True)
```

[source]

Create a spreadsheet-style pivot table as a DataFrame.

The levels in the pivot table will be stored in Multiindex objects (hierarchical indexes) on the index and columns of the result DataFrame.

Parameters: **data** : DataFrame

values : column to aggregate, optional

index : column, Grouper, array, or list of the previous

If an array is passed, it must be the same length as the data. The list can contain any of the other types (except list). Keys to group by on the pivot table index. If an array is passed, it is being used as the same manner as column values.

columns : column, Grouper, array, or list of the previous

If an array is passed, it must be the same length as the data. The list can contain any of the other types (except list). Keys to group by on the pivot table column. If an array is passed, it is being used as the same manner as column values.

aggfunc : function, list of functions, dict, default numpy.mean

If list of functions passed, the resulting pivot table will have hierarchical columns whose top level are the function names (inferred from the function objects themselves) If dict is passed, the key is column to aggregate and value is function or list of functions.

fill_value : scalar, default None

Value to replace missing values with (in the resulting pivot table, after aggregation).

margins : bool, default False

Add all row / columns (e.g. for subtotal / grand totals).

dropna : bool, default True

Do not include columns whose entries are all NaN.

margins_name : str, default 'All'

Name of the row / column that will contain the totals when margins is True.

observed : bool, default False

This only applies if any of the groupers are Categoricals. If True: only show observed values for categorical groupers. If False: show all values for categorical groupers.

sort : bool, default True

Specifies if the result should be sorted.

New in version 1.3.0.

Returns: DataFrame

An Excel style pivot table.

Examples

```
>>> df = pd.DataFrame({"A": ["foo", "foo", "foo", "foo", "foo",
...                          "bar", "bar", "bar", "bar"],
...                   "B": ["one", "one", "one", "two", "two",
...                          "one", "one", "two", "two"],
...                   "C": ["small", "large", "large", "small",
...                          "small", "large", "small", "small",
...                          "large"],
...                   "D": [1, 2, 2, 3, 3, 4, 5, 6, 7],
...                   "E": [2, 4, 5, 5, 6, 6, 8, 9, 9]})
>>> df
   A  B    C  D  E
0  foo one small 1  2
1  foo one large 2  4
2  foo one large 2  5
3  foo two small 3  5
4  foo two small 3  6
5  bar one large 4  6
6  bar one small 5  8
7  bar two small 6  9
8  bar two large 7  9
```

This first example aggregates values by taking the sum.

```
>>> table = pd.pivot_table(df, values='D', index=['A', 'B'],
...                         columns=['C'], aggfunc=np.sum)
>>> table
C      large  small
A  B
bar one    4.0    5.0
   two    7.0    6.0
foo one    4.0    1.0
   two    NaN    6.0
```

We can also fill missing values using the *fill_value* parameter.

```
>>> table = pd.pivot_table(df, values='D', index=['A', 'B'],
...                         columns=['C'], aggfunc=np.sum, fill_value=0)
>>> table
C      large  small
A  B
bar one     4     5
   two     7     6
foo one     4     1
   two     0     6
```




Plot by Pandas

pandas.DataFrame.plot

`DataFrame.plot(*args, **kwargs)` [\[source\]](#)

Make plots of Series or DataFrame.

Uses the backend specified by the option `plotting.backend`. By default, matplotlib is used.

Parameters: **data** : *Series or DataFrame*

The object for which the method is called.

x : *label or position, default None*

Only used if data is a DataFrame.

y : *label, position or list of label, positions, default None*

Allows plotting of one column versus another. Only used if data is a DataFrame.

kind : *str*

The kind of plot to produce:

- 'line': line plot (default)
- 'bar': vertical bar plot
- 'barh': horizontal bar plot
- 'hist': histogram
- 'box': boxplot
- 'kde': Kernel Density Estimation plot
- 'density': same as 'kde'
- 'area': area plot
- 'pie': pie plot
- 'scatter': scatter plot (DataFrame only)
- 'hexbin': hexbin plot (DataFrame only)

ax : *matplotlib axes object, default None*

An axes of the current figure.

subplots : *bool, default False*

Make separate subplots for each column.

sharex : *bool, default True if ax is None else False*

In case `subplots=True`, share x axis and set some x axis labels to invisible; defaults to True if ax is None otherwise False if an ax is passed in; Be aware, that passing in both an ax and `sharex=True` will alter all x axis labels for all axes in a figure.

sharey : *bool, default False*

In case `subplots=True`, share y axis and set some y axis labels to invisible.

layout : *tuple, optional*

(rows, columns) for the layout of subplots.

figsize : *a tuple (width, height) in inches*

Size of a figure object.

use_index : *bool, default True*

Use index as ticks for x axis.

title : *str or list*

Title to use for the plot. If a string is passed, print the string at the top of the figure. If a list is passed and `subplots` is True, print each item in the list above the corresponding subplot.

grid : *bool, default None (matplotlib style default)*

Axis grid lines.

legend : *bool or {'reverse'}*

Place legend on axis subplots.

style : *list or dict*

The matplotlib line style per column.

logx : *bool or 'sym', default False*

Use log scaling or symlog scaling on x axis. .. versionchanged:: 0.25.0

logy : *bool or 'sym', default False*

Use log scaling or symlog scaling on y axis. .. versionchanged:: 0.25.0

loglog : *bool or 'sym', default False*

Use log scaling or symlog scaling on both x and y axes. .. versionchanged:: 0.25.0

xticks : *sequence*

Values to use for the xticks.

yticks : *sequence*

Values to use for the yticks.

xlim : *2-tuple/list*

Set the x limits of the current axes.

ylim : *2-tuple/list*

Set the y limits of the current axes.

xlabel : *label, optional*

Name to use for the xlabel on x-axis. Default uses index name as xlabel, or the x-column name for planar plots.

🆕 New in version 1.1.0.

🔄 Changed in version 1.2.0: Now applicable to planar plots (scatter, hexbin).

ylabel : *label, optional*

Name to use for the ylabel on y-axis. Default will show no ylabel, or the y-column name for planar plots.

🆕 New in version 1.1.0.

🔄 Changed in version 1.2.0: Now applicable to planar plots (scatter, hexbin).

rot : *int, default None*

Rotation for ticks (xticks for vertical, yticks for horizontal plots).

fontsize : *int, default None*

Font size for xticks and yticks.

colormap : *str or matplotlib colormap object, default None*

Colormap to select colors from. If string, load colormap with that name from matplotlib.

colorbar : *bool, optional*

If True, plot colorbar (only relevant for 'scatter' and 'hexbin' plots).

position : *float*

Specify relative alignments for bar plot layout. From 0 (left/bottom-end) to 1 (right/top-end). Default is 0.5 (center).

table : *bool, Series or DataFrame, default False*

If True, draw a table using the data in the DataFrame and the data will be transposed to meet matplotlib's default layout. If a Series or DataFrame is passed, use passed data to draw a table.

yerr : *DataFrame, Series, array-like, dict and str*

See [Plotting with Error Bars](#) for detail.

xerr : *DataFrame, Series, array-like, dict and str*

Equivalent to yerr.

stacked : *bool, default False in line and bar plots, and True in area plot*

If True, create stacked plot.

sort_columns : *bool, default False*

Sort column names to determine plot ordering.

secondary_y : *bool or sequence, default False*

Whether to plot on the secondary y-axis if a list/tuple, which columns to plot on secondary y-axis.

mark_right : *bool, default True*

When using a secondary_y axis, automatically mark the column labels with "(right)" in the legend.

include_bool : *bool, default is False*

If True, boolean values can be plotted.

backend : *str, default None*

Backend to use instead of the backend specified in the option `plotting.backend`.

For instance, 'matplotlib'. Alternatively, to specify the `plotting.backend` for the whole session, set `pd.options.plotting.backend`.

🆕 New in version 1.0.0.

****kwargs**

Options to pass to matplotlib plotting method.

Returns:

`matplotlib.axes.Axes` or `numpy.ndarray` of them

If the backend is not the default matplotlib one, the return value will be the object returned by the backend.



파일 입출력 (CSV)

pandas.read_csv

```
pandas.read_csv(filepath_or_buffer, sep=NoDefault.no_default, delimiter=None,
header='infer', names=NoDefault.no_default, index_col=None, usecols=None,
squeeze=None, prefix=NoDefault.no_default, mangle_dupe_cols=True, dtype=None,
engine=None, converters=None, true_values=None, false_values=None,
skipinitialspace=False, skiprows=None, skipfooter=0, nrows=None, na_values=None,
keep_default_na=True, na_filter=True, verbose=False, skip_blank_lines=True,
parse_dates=None, infer_datetime_format=False, keep_date_col=False,
date_parser=None, dayfirst=False, cache_dates=True, iterator=False,
chunksize=None, compression='infer', thousands=None, decimal='.',
lineterminator=None, quotechar='"', quoting=0, doublequote=True, escapechar=None,
comment=None, encoding=None, encoding_errors='strict', dialect=None,
error_bad_lines=None, warn_bad_lines=None, on_bad_lines=None,
delim_whitespace=False, low_memory=True, memory_map=False, float_precision=None,
storage_options=None) \[source\]
```

Read a comma-separated values (csv) file into DataFrame.

Also supports optionally iterating or breaking of the file into chunks.

Additional help can be found in the online docs for [IO Tools](#).

pandas.DataFrame.to_csv

```
DataFrame.to_csv(path_or_buf=None, sep=',', na_rep='', float_format=None,
columns=None, header=True, index=True, index_label=None, mode='w', encoding=None,
compression='infer', quoting=None, quotechar='"', line_terminator=None,
chunksize=None, date_format=None, doublequote=True, escapechar=None, decimal='.',
errors='strict', storage_options=None) \[source\]
```

Write object to a comma-separated values (csv) file.





파일 입출력 (Parquet)

Columnar vs Row-based

Here is an example of translating a logical table schema. First the example table:

A	B	C
A1	B1	C1
A2	B2	C2
A3	B3	C3

In a row-based layout each row follows the next:

A1	B1	C1	A2	B2	C2	A3	B3	C3
----	----	----	----	----	----	----	----	----

While for a column-oriented layout it stores one column after the next:

A1	A2	A3	B1	B2	B3	C1	C2	C3
----	----	----	----	----	----	----	----	----

열 기반 압축(Parquet)



행 기반 압축(전통적 방식)



열을 기반으로 데이터를 처리하면 행 기반으로 압축했을때에 비해 데이터의 압축률이 더 높고, 필요한 열의 데이터만 읽어서 처리하는 것이 가능하기 때문에 데이터 처리에 들어가는 자원을 절약할 수 있습니다.

파케이는 하둡 에코시스템 안에서 언제든지 사용 가능한 데이터 저장 포맷으로, 스파크 SQL은 자동으로 기존의 데이터 스키마를 유지하는 parquet 파일의 읽기와 쓰기를 지원합니다. 유사한 파일 형식으로 ORC가 있는데요, ORC도 파케이와 마찬가지로 압축률이 높고 스키마를 가지고있으며 처리속도가 빠르지만 하이버에서만 사용할 수 있었습니다. (현재는 아님) 회사가 다르다보니 통합된 형태로 발전되지 못하고 있다가 트위터에서 Parquet을 발표하였습니다. ORC는 Hive에 최적화된 형식이고, Parquet은 스파크에 최적화된 형식이라고 보시면 좋을 것 같습니다.

Parquet 파일을 쓸 때 모든 길림은 호환성을 위해 자동으로 null을 허용하도록 변경됩니다.



파일 입출력 (Parquet)

pandas.read_parquet

```
pandas.read_parquet(path, engine='auto', columns=None, storage_options=None,
use_nullable_dtypes=False, **kwargs) 1 [source]
```

Load a parquet object from the file path, returning a DataFrame.

pandas.DataFrame.to_parquet

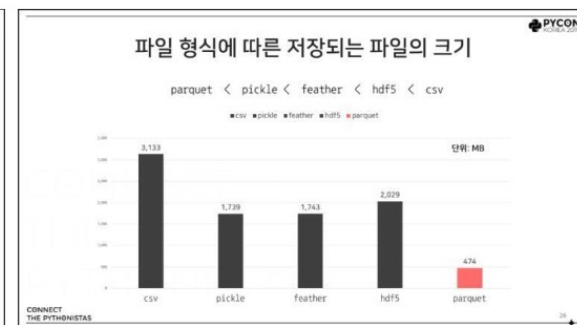
```
DataFrame.to_parquet(path=None, engine='auto', compression='snappy', index=None,
partition_cols=None, storage_options=None, **kwargs) [source]
```

Write a DataFrame to the binary parquet format.

This function writes the dataframe as a [parquet file](#). You can choose different parquet backends, and have the option of compression. See [the user guide](#) for more details.

장점

- 높은 압축률. 칼럼 단위로 구성하면 데이터가 더 균일하므로 압축률이 높아진다.
- 데이터를 전체 칼럼중에서 일부 칼럼을 선택해서 가져오는 형식이므로 선택되지 않은 칼럼의 데이터에서는 I/O가 발생하지 않게된다.
- 칼럼에 동일한 데이터 타입이 저장되기 때문에 칼럼별로 적합한(데이터형에 유리한) 인코딩을 사용할 수 있다.



Q&A and Break Time

질의응답 및 휴식시간 (5분)



2부: 고급 *pandas* 및 적용 사례

- 1) Categorical 데이터
- 2) 고급 GroupBy 사용
- 3) 메서드 연결 기법





Categorical 데이터 > 개발 배경과 동기

Categorical Data > Background and Motivation

12.1 Categorical Data

This section introduces the pandas Categorical type. I will show how you can achieve better performance and memory use in some pandas operations by using it. I also introduce some tools for using categorical data in statistics and machine learning applications.

Background and Motivation

Frequently, a column in a table may contain repeated instances of a smaller set of distinct values. We have already seen functions like `unique` and `value_counts`, which enable us to extract the distinct values from an array and compute their frequencies, respectively:

```
In [10]: import numpy as np; import pandas as pd

In [11]: values = pd.Series(['apple', 'orange', 'apple',
    ....:                    'apple'] * 2)

In [12]: values
Out[12]:
0    apple
1   orange
2    apple
3    apple
4    apple
5   orange
6    apple
7    apple
dtype: object
```

```
In [13]: pd.unique(values)
Out[13]: array(['apple', 'orange'], dtype=object)
```

```
In [14]: pd.value_counts(values)
Out[14]:
apple    6
orange   2
dtype: int64
```

Many data systems (for data warehousing, statistical computing, or other uses) have developed specialized approaches for representing data with repeated values for more efficient storage and computation. In data warehousing, a best practice is to use so-called *dimension tables* containing the distinct values and storing the primary observations as integer keys referencing the dimension table:

```
In [15]: values = pd.Series([0, 1, 0, 0] * 2)
```

```
In [16]: dim = pd.Series(['apple', 'orange'])
```

```
In [17]: values
Out[17]:
0    0
1    1
2    0
3    0
4    0
5    1
6    0
7    0
dtype: int64
```

```
In [18]: dim
Out[18]:
0    apple
1   orange
dtype: object
```





Categorical 데이터 > 개발 배경과 동기

Categorical Data > Background and Motivation

We can use the `take` method to restore the original Series of strings:

```
In [19]: dim.take(values)
Out[19]:
0    apple
1   orange
0    apple
0    apple
0    apple
1   orange
0    apple
0    apple
dtype: object
```

This representation as integers is called the *categorical* or *dictionary-encoded* representation. The array of distinct values can be called the *categories*, *dictionary*, or *levels* of the data. In this book we will use the terms *categorical* and *categories*. The integer values that reference the categories are called the *category codes* or simply *codes*.

The categorical representation can yield significant performance improvements when you are doing analytics. You can also perform transformations on the categories while leaving the codes unmodified. Some example transformations that can be made at relatively low cost are:

- Renaming categories
- Appending a new category without changing the order or position of the existing categories





Categorical 데이터 > pandas의 Categorical

Categorical Data > Categorical Type in pandas

Categorical Type in pandas

pandas has a special `Categorical` type for holding data that uses the integer-based categorical representation or *encoding*. Let's consider the example Series from before:

```
In [20]: fruits = ['apple', 'orange', 'apple', 'apple'] * 2
```

```
In [21]: N = len(fruits)
```

```
In [22]: df = pd.DataFrame({'fruit': fruits,
.....:                     'basket_id': np.arange(N),
.....:                     'count': np.random.randint(3, 15, size=N),
.....:                     'weight': np.random.uniform(0, 4, size=N)},
.....:                     columns=['basket_id', 'fruit', 'count', 'weight'])
```

```
In [23]: df
```

```
Out[23]:
```

	basket_id	fruit	count	weight
0	0	apple	5	3.858058
1	1	orange	8	2.612708
2	2	apple	4	2.995627
3	3	apple	7	2.614279
4	4	apple	12	2.990859
5	5	orange	8	3.845227
6	6	apple	5	0.033553
7	7	apple	4	0.425778

Here, `df['fruit']` is an array of Python string objects. We can convert it to categorical by calling:

```
In [24]: fruit_cat = df['fruit'].astype('category')
```

```
In [25]: fruit_cat
```

```
Out[25]:
```

0	apple
1	orange
2	apple
3	apple
4	apple
5	orange
6	apple
7	apple

```
Name: fruit, dtype: category
Categories (2, object): [apple, orange]
```

The values for `fruit_cat` are not a NumPy array, but an instance of `pandas.Categorical`:

```
In [26]: c = fruit_cat.values
```

```
In [27]: type(c)
```

```
Out[27]: pandas.core.categorical.Categorical
```




Categorical 데이터 > pandas의 Categorical

Categorical Data > Categorical Type in pandas

The Categorical object has `categories` and `codes` attributes:

```
In [28]: c.categories
Out[28]: Index(['apple', 'orange'], dtype='object')
```

```
In [29]: c.codes
Out[29]: array([0, 1, 0, 0, 0, 1, 0, 0], dtype=int8)
```

You can convert a DataFrame column to categorical by assigning the converted result:

```
In [30]: df['fruit'] = df['fruit'].astype('category')
```

```
In [31]: df.fruit
Out[31]:
0    apple
1    orange
2    apple
3    apple
4    apple
5    orange
6    apple
7    apple
Name: fruit, dtype: category
Categories (2, object): [apple, orange]
```

You can also create `pandas.Categorical` directly from other types of Python sequences:

```
In [32]: my_categories = pd.Categorical(['foo', 'bar', 'baz', 'foo', 'bar'])

In [33]: my_categories
Out[33]:
[foo, bar, baz, foo, bar]
Categories (3, object): [bar, baz, foo]
```

If you have obtained categorical encoded data from another source, you can use the alternative `from_codes` constructor:

```
In [34]: categories = ['foo', 'bar', 'baz']

In [35]: codes = [0, 1, 2, 0, 0, 1]

In [36]: my_cats_2 = pd.Categorical.from_codes(codes, categories)

In [37]: my_cats_2
Out[37]:
[foo, bar, baz, foo, foo, bar]
Categories (3, object): [foo, bar, baz]
```

Unless explicitly specified, categorical conversions assume no specific ordering of the categories. So the `categories` array may be in a different order depending on the ordering of the input data. When using `from_codes` or any of the other constructors, you can indicate that the categories have a meaningful ordering:

```
In [38]: ordered_cat = pd.Categorical.from_codes(codes, categories,
....:                                             ordered=True)

In [39]: ordered_cat
Out[39]:
[foo, bar, baz, foo, foo, bar]
Categories (3, object): [foo < bar < baz]
```

The output `[foo < bar < baz]` indicates that 'foo' precedes 'bar' in the ordering, and so on. An unordered categorical instance can be made ordered with `as_ordered`:

```
In [40]: my_cats_2.as_ordered()
Out[40]:
[foo, bar, baz, foo, foo, bar]
Categories (3, object): [foo < bar < baz]
```

As a last note, categorical data need not be strings, even though I have only showed string examples. A categorical array can consist of any immutable value types.



Categorical 데이터 > Categorical 연산

Categorical Data > Computations with Categoricals

Computations with Categoricals

Using `Categorical` in pandas compared with the non-encoded version (like an array of strings) generally behaves the same way. Some parts of pandas, like the `groupby` function, perform better when working with categoricals. There are also some functions that can utilize the `ordered` flag.

Let's consider some random numeric data, and use the `pandas.qcut` binning function. This return `pandas.Categorical`; we used `pandas.cut` earlier in the book but glossed over the details of how categoricals work:

```
In [41]: np.random.seed(12345)
```

```
In [42]: draws = np.random.randn(1000)
```

```
In [43]: draws[:5]
```

```
Out[43]: array([-0.2047,  0.4789, -0.5194, -0.5557,  1.9658])
```

Let's compute a quartile binning of this data and extract some statistics:

```
In [44]: bins = pd.qcut(draws, 4)
```

```
In [45]: bins
```

```
Out[45]:
```

```
[(-0.684, -0.0101], (-0.0101, 0.63], (-0.684, -0.0101], (-0.684, -0.0101], (0.63, 3.928], ..., (-0.0101, 0.63], (-0.684, -0.0101], (-2.95, -0.684], (-0.0101, 0.63], (0.63, 3.928]]
```

```
Length: 1000
```

```
Categories (4, interval[float64]): [(-2.95, -0.684] < (-0.684, -0.0101] < (-0.0101, 0.63] <
```

```
(0.63, 3.928]]
```

While useful, the exact sample quartiles may be less useful for producing a report than quartile names. We can achieve this with the `labels` argument to `qcut`:

```
In [46]: bins = pd.qcut(draws, 4, labels=['Q1', 'Q2', 'Q3', 'Q4'])
```

```
In [47]: bins
```

```
Out[47]:
```

```
[Q2, Q3, Q2, Q2, Q4, ..., Q3, Q2, Q1, Q3, Q4]
```

```
Length: 1000
```

```
Categories (4, object): [Q1 < Q2 < Q3 < Q4]
```

```
In [48]: bins.codes[:10]
```

```
Out[48]: array([1, 2, 1, 1, 3, 3, 2, 2, 3, 3], dtype=int8)
```

The labeled bins categorical does not contain information about the bin edges in the data, so we can use `groupby` to extract some summary statistics:

```
In [49]: bins = pd.Series(bins, name='quartile')
```

```
In [50]: results = (pd.Series(draws)
```

```
.....:         .groupby(bins)
```

```
.....:         .agg(['count', 'min', 'max'])
```

```
.....:         .reset_index())
```

```
In [51]: results
```

```
Out[51]:
```

	quartile	count	min	max
0	Q1	250	-2.949343	-0.685484
1	Q2	250	-0.683066	-0.010115
2	Q3	250	-0.010032	0.628894
3	Q4	250	0.634238	3.927528



Categorical 데이터 > Categorical 연산

Categorical Data > Computations with Categoricals

The 'quartile' column in the result retains the original categorical information, including ordering, from bins:

```
In [52]: results['quartile']
Out[52]:
0    Q1
1    Q2
2    Q3
3    Q4
Name: quartile, dtype: category
Categories (4, object): [Q1 < Q2 < Q3 < Q4]
```

Better performance with categoricals

If you do a lot of analytics on a particular dataset, converting to categorical can yield substantial overall performance gains. A categorical version of a DataFrame column will often use significantly less memory, too. Let's consider some Series with 10 million elements and a small number of distinct categories:

```
In [53]: N = 10000000

In [54]: draws = pd.Series(np.random.randn(N))

In [55]: labels = pd.Series(['foo', 'bar', 'baz', 'qux'] * (N // 4))
```

Now we convert labels to categorical:

```
In [56]: categories = labels.astype('category')
```

Now we note that labels uses significantly more memory than categories:

```
In [57]: labels.memory_usage()
Out[57]: 800000000

In [58]: categories.memory_usage()
Out[58]: 10000272
```

The conversion to category is not free, of course, but it is a one-time cost:

```
In [59]: %time _ = labels.astype('category')
CPU times: user 490 ms, sys: 240 ms, total: 730 ms
Wall time: 726 ms
```

GroupBy operations can be significantly faster with categoricals because the underlying algorithms use the integer-based codes array instead of an array of strings.



Categorical 데이터 > Categorical 메서드

Categorical Data > Categorical Methods

Categorical Methods

Series containing categorical data have several special methods similar to the `Series.str` specialized string methods. This also provides convenient access to the categories and codes. Consider the Series:

```
In [60]: s = pd.Series(['a', 'b', 'c', 'd'] * 2)
```

```
In [61]: cat_s = s.astype('category')
```

```
In [62]: cat_s
```

```
Out[62]:
```

```
0    a
```

```
1    b
```

```
2    c
```

```
3    d
```

```
4    a
```

```
5    b
```

```
6    c
```

```
7    d
```

```
dtype: category
```

```
Categories (4, object): [a, b, c, d]
```

The special attribute `cat` provides access to categorical methods:

```
In [63]: cat_s.cat.codes
```

```
Out[63]:
```

```
0    0
```

```
1    1
```

```
2    2
```

```
3    3
```

```
4    0
```

```
5    1
```

```
6    2
```

```
7    3
```

```
dtype: int8
```

```
In [64]: cat_s.cat.categories
```

```
Out[64]: Index(['a', 'b', 'c', 'd'], dtype='object')
```





Categorical 데이터 > Categorical 메서드

Categorical Data > Categorical Methods

Suppose that we know the actual set of categories for this data extends beyond the four values observed in the data. We can use the `set_categories` method to change them:

```
In [65]: actual_categories = ['a', 'b', 'c', 'd', 'e']

In [66]: cat_s2 = cat_s.cat.set_categories(actual_categories)

In [67]: cat_s2
Out[67]:
0    a
1    b
2    c
3    d
4    a
5    b
6    c
7    d
dtype: category
Categories (5, object): [a, b, c, d, e]
```

While it appears that the data is unchanged, the new categories will be reflected in operations that use them. For example, `value_counts` respects the categories, if present:

```
In [68]: cat_s.value_counts()
Out[68]:
d    2
c    2
b    2
a    2
dtype: int64

In [69]: cat_s2.value_counts()
Out[69]:
d    2
c    2
b    2
a    2
e    0
dtype: int64
```

In large datasets, categoricals are often used as a convenient tool for memory savings and better performance. After you filter a large DataFrame or Series, many of the categories may not appear in the data. To help with this, we can use the `remove_unused_categories` method to trim unobserved categories:

```
In [70]: cat_s3 = cat_s[cat_s.isin(['a', 'b'])]

In [71]: cat_s3
Out[71]:
0    a
1    b
4    a
5    b
dtype: category
Categories (4, object): [a, b, c, d]

In [72]: cat_s3.cat.remove_unused_categories()
Out[72]:
0    a
1    b
4    a
5    b
dtype: category
Categories (2, object): [a, b]
```

See [Table 12-1](#) for a listing of available categorical methods.

Table 12-1. Categorical methods for Series in pandas

Method	Description
<code>add_categories</code>	Append new (unused) categories at end of existing categories
<code>as_ordered</code>	Make categories ordered
<code>as_unordered</code>	Make categories unordered
<code>remove_categories</code>	Remove categories, setting any removed values to null
<code>remove_unused_categories</code>	Remove any category values which do not appear in the data
<code>rename_categories</code>	Replace categories with indicated set of new category names; cannot change the number of categories
<code>reorder_categories</code>	Behaves like <code>rename_categories</code> , but can also change the result to have ordered categories
<code>set_categories</code>	Replace the categories with the indicated set of new categories; can add or remove categories



Categorical 데이터 > Categorical 메서드

Categorical Data > Categorical Methods

Creating dummy variables for modeling

When you're using statistics or machine learning tools, you'll often transform categorical data into *dummy variables*, also known as *one-hot* encoding. This involves creating a DataFrame with a column for each distinct category; these columns contain 1s for occurrences of a given category and 0 otherwise.

Consider the previous example:

```
In [73]: cat_s = pd.Series(['a', 'b', 'c', 'd'] * 2, dtype='category')
```

As mentioned previously in [Chapter 7](#), the `pandas.get_dummies` function converts this one-dimensional categorical data into a DataFrame containing the dummy variable:

```
In [74]: pd.get_dummies(cat_s)
```

```
Out[74]:
```

	a	b	c	d
0	1	0	0	0
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1
4	1	0	0	0
5	0	1	0	0
6	0	0	1	0
7	0	0	0	1





고급 GroupBy 사용 > 그룹 변환과 GroupBy 객체 풀어내기

Advanced GroupBy Use > Group Transforms and “Unwrapped” GroupBys

12.2 Advanced GroupBy Use

While we’ve already discussed using the groupby method for Series and DataFrame in depth in [Chapter 10](#), there are some additional techniques that you may find of use.

Group Transforms and “Unwrapped” GroupBys

In [Chapter 10](#) we looked at the apply method in grouped operations for performing transformations. There is another built-in method called transform, which is similar to apply but imposes more constraints on the kind of function you can use:

- It can produce a scalar value to be broadcast to the shape of the group
- It can produce an object of the same shape as the input group
- It must not mutate its input

Let’s consider a simple example for illustration:

```
In [75]: df = pd.DataFrame({'key': ['a', 'b', 'c'] * 4,  
.....:                    'value': np.arange(12.)})
```

```
In [76]: df  
Out[76]:  
   key  value  
0    a    0.0  
1    b    1.0  
2    c    2.0  
3    a    3.0  
4    b    4.0  
5    c    5.0  
6    a    6.0  
7    b    7.0  
8    c    8.0  
9    a    9.0  
10   b   10.0  
11   c   11.0
```

Here are the group means by key:

```
In [77]: g = df.groupby('key').value  
  
In [78]: g.mean()  
Out[78]:  
key  
a    4.5  
b    5.5  
c    6.5  
Name: value, dtype: float64
```




고급 GroupBy 사용 > 그룹 변환과 GroupBy 객체 풀어내기

Advanced GroupBy Use > Group Transforms and "Unwrapped" GroupBys

Suppose instead we wanted to produce a Series of the same shape as `df['value']` but with values replaced by the average grouped by 'key'. We can pass the function `lambda x: x.mean()` to transform:

```
In [79]: g.transform(lambda x: x.mean())
Out[79]:
0    4.5
1    5.5
2    6.5
3    4.5
4    5.5
5    6.5
6    4.5
7    5.5
8    6.5
9    4.5
10   5.5
11   6.5
Name: value, dtype: float64
```

For built-in aggregation functions, we can pass a string alias as with the GroupBy `agg` method:

```
In [80]: g.transform('mean')
Out[80]:
0    4.5
1    5.5
2    6.5
3    4.5
4    5.5
5    6.5
6    4.5
7    5.5
8    6.5
9    4.5
10   5.5
11   6.5
Name: value, dtype: float64
```

Like `apply`, `transform` works with functions that return Series, but the result must be the same size as the input. For example, we can multiply each group by 2 using a lambda function:

```
In [81]: g.transform(lambda x: x * 2)
Out[81]:
0    8.0
1    2.0
2    4.0
3    6.0
4    8.0
5   10.0
6   12.0
7   14.0
8   16.0
9   18.0
10  20.0
11  22.0
Name: value, dtype: float64
```

As a more complicated example, we can compute the ranks in descending order for each group:

```
In [82]: g.transform(lambda x: x.rank(ascending=False))
Out[82]:
0    4.0
1    4.0
2    4.0
3    3.0
4    3.0
5    3.0
6    2.0
7    2.0
8    2.0
9    1.0
10   1.0
11   1.0
Name: value, dtype: float64
```



고급 GroupBy 사용 > 그룹 변환과 GroupBy 객체 풀어내기

Advanced GroupBy Use > Group Transforms and “Unwrapped” GroupBys

Consider a group transformation function composed from simple aggregations:

```
def normalize(x):  
    return (x - x.mean()) / x.std()
```

We can obtain equivalent results in this case either using transform or apply:

```
In [84]: g.transform(normalize)  
Out[84]:  
0    -1.161895  
1    -1.161895  
2    -1.161895  
3    -0.387298  
4    -0.387298  
5    -0.387298  
6     0.387298  
7     0.387298  
8     0.387298  
9     1.161895  
10    1.161895  
11    1.161895  
Name: value, dtype: float64
```

```
In [85]: g.apply(normalize)  
Out[85]:  
0    -1.161895  
1    -1.161895  
2    -1.161895  
3    -0.387298  
4    -0.387298  
5    -0.387298  
6     0.387298  
7     0.387298  
8     0.387298  
9     1.161895  
10    1.161895  
11    1.161895  
Name: value, dtype: float64
```

Built-in aggregate functions like 'mean' or 'sum' are often much faster than a general apply function. These also have a “fast path” when used with transform. This allows us to perform a so-called *unwrapped* group operation:

```
In [86]: g.transform('mean')  
Out[86]:  
0     4.5  
1     5.5  
2     6.5  
3     4.5  
4     5.5  
5     6.5  
6     4.5  
7     5.5  
8     6.5  
9     4.5  
10    5.5  
11    6.5  
Name: value, dtype: float64
```

```
In [87]: normalized = (df['value'] - g.transform('mean')) / g.transform('std')
```

```
In [88]: normalized  
Out[88]:  
0    -1.161895  
1    -1.161895  
2    -1.161895  
3    -0.387298  
4    -0.387298  
5    -0.387298  
6     0.387298  
7     0.387298  
8     0.387298  
9     1.161895  
10    1.161895  
11    1.161895  
Name: value, dtype: float64
```

While an unwrapped group operation may involve multiple group aggregations, the overall benefit of vectorized operations often outweighs this.



고급 GroupBy 사용 > 시계열 그룹 리샘플링

Advanced GroupBy Use > Grouped Time Resampling

Grouped Time Resampling

For time series data, the `resample` method is semantically a group operation based on a time intervalization. Here's a small example table:

```
In [89]: N = 15
```

```
In [90]: times = pd.date_range('2017-05-20 00:00', freq='1min', periods=N)
```

```
In [91]: df = pd.DataFrame({'time': times,
.....:                    'value': np.arange(N)})
```

```
In [92]: df
```

```
Out[92]:
```

	time	value
0	2017-05-20 00:00:00	0
1	2017-05-20 00:01:00	1
2	2017-05-20 00:02:00	2
3	2017-05-20 00:03:00	3
4	2017-05-20 00:04:00	4
5	2017-05-20 00:05:00	5
6	2017-05-20 00:06:00	6
7	2017-05-20 00:07:00	7
8	2017-05-20 00:08:00	8
9	2017-05-20 00:09:00	9
10	2017-05-20 00:10:00	10
11	2017-05-20 00:11:00	11
12	2017-05-20 00:12:00	12
13	2017-05-20 00:13:00	13
14	2017-05-20 00:14:00	14

Here, we can index by 'time' and then resample:

```
In [93]: df.set_index('time').resample('5min').count()
```

```
Out[93]:
```

	value
time	
2017-05-20 00:00:00	5
2017-05-20 00:05:00	5
2017-05-20 00:10:00	5

Suppose that a DataFrame contains multiple time series, marked by an additional group key column:

```
In [94]: df2 = pd.DataFrame({'time': times.repeat(3),
.....:                    'key': np.tile(['a', 'b', 'c'], N),
.....:                    'value': np.arange(N * 3.)})
```

```
In [95]: df2[:7]
```

```
Out[95]:
```

	key	time	value
0	a	2017-05-20 00:00:00	0.0
1	b	2017-05-20 00:00:00	1.0
2	c	2017-05-20 00:00:00	2.0
3	a	2017-05-20 00:01:00	3.0
4	b	2017-05-20 00:01:00	4.0
5	c	2017-05-20 00:01:00	5.0
6	a	2017-05-20 00:02:00	6.0



고급 GroupBy 사용 > 시계열 그룹 리샘플링

Advanced GroupBy Use > Grouped Time Resampling

To do the same resampling for each value of 'key', we introduce the `pandas.Time Grouper` object:

```
In [96]: time_key = pd.TimeGrouper('5min')
```

We can then set the time index, group by 'key' and `time_key`, and aggregate:

```
In [97]: resampled = (df2.set_index('time')
.....:                  .groupby(['key', time_key])
.....:                  .sum())
```

```
In [98]: resampled
```

```
Out[98]:
```

		value
key	time	
a	2017-05-20 00:00:00	30.0
	2017-05-20 00:05:00	105.0
	2017-05-20 00:10:00	180.0
b	2017-05-20 00:00:00	35.0
	2017-05-20 00:05:00	110.0
	2017-05-20 00:10:00	185.0
c	2017-05-20 00:00:00	40.0
	2017-05-20 00:05:00	115.0
	2017-05-20 00:10:00	190.0

```
In [99]: resampled.reset_index()
```

```
Out[99]:
```

	key	time	value
0	a	2017-05-20 00:00:00	30.0
1	a	2017-05-20 00:05:00	105.0
2	a	2017-05-20 00:10:00	180.0
3	b	2017-05-20 00:00:00	35.0
4	b	2017-05-20 00:05:00	110.0
5	b	2017-05-20 00:10:00	185.0
6	c	2017-05-20 00:00:00	40.0
7	c	2017-05-20 00:05:00	115.0
8	c	2017-05-20 00:10:00	190.0

One constraint with using `TimeGrouper` is that the time must be the index of the Series or DataFrame.



12.3 Techniques for Method Chaining

When applying a sequence of transformations to a dataset, you may find yourself creating numerous temporary variables that are never used in your analysis. Consider this example, for instance:

```
df = load_data()
df2 = df[df['col2'] < 0]
df2['col1_demeaned'] = df2['col1'] - df2['col1'].mean()
result = df2.groupby('key').col1_demeaned.std()
```

While we're not using any real data here, this example highlights some new methods. First, the `DataFrame.assign` method is a *functional* alternative to column assignments of the form `df[k] = v`. Rather than modifying the object in-place, it returns a new `DataFrame` with the indicated modifications. So these statements are equivalent:

```
# Usual non-functional way
df2 = df.copy()
df2['k'] = v

# Functional assign way
df2 = df.assign(k=v)
```

Assigning in-place may execute faster than using `assign`, but `assign` enables easier method chaining:

```
result = (df2.assign(col1_demeaned=df2.col1 - df2.col2.mean())
          .groupby('key')
          .col1_demeaned.std())
```

I used the outer parentheses to make it more convenient to add line breaks.

One thing to keep in mind when doing method chaining is that you may need to refer to temporary objects. In the preceding example, we cannot refer to the result of `load_data` until it has been assigned to the temporary variable `df`. To help with this, `assign` and many other pandas functions accept function-like arguments, also known as *callables*.

To show callables in action, consider a fragment of the example from before:

```
df = load_data()
df2 = df[df['col2'] < 0]
```

This can be rewritten as:

```
df = (load_data()
      [lambda x: x['col2'] < 0])
```





메서드 연결 기법

Techniques for Method Chaining > The pipe Method

Here, the result of `load_data` is not assigned to a variable, so the function passed into `[]` is then *bound* to the object at that stage of the method chain.

We can continue, then, and write the entire sequence as a single chained expression:

```
result = (load_data()  
         [lambda x: x.col2 < 0]  
         .assign(col1_demeaned=lambda x: x.col1 - x.col1.mean())  
         .groupby('key')  
         .col1_demeaned.std())
```

Whether you prefer to write code in this style is a matter of taste, and splitting up the expression into multiple steps may make your code more readable.





메서드 연결 기법 > pipe 메서드

Techniques for Method Chaining > The pipe Method

The pipe Method

You can accomplish a lot with built-in pandas functions and the approaches to method chaining with callables that we just looked at. However, sometimes you need to use your own functions or functions from third-party libraries. This is where the pipe method comes in.

Consider a sequence of function calls:

```
a = f(df, arg1=v1)
b = g(a, v2, arg3=v3)
c = h(b, arg4=v4)
```

When using functions that accept and return Series or DataFrame objects, you can rewrite this using calls to pipe:

```
result = (df.pipe(f, arg1=v1)
          .pipe(g, v2, arg3=v3)
          .pipe(h, arg4=v4))
```

The statement `f(df)` and `df.pipe(f)` are equivalent, but pipe makes chained invocation easier.

A potentially useful pattern for pipe is to generalize sequences of operations into reusable functions. As an example, let's consider subtracting group means from a column:

```
g = df.groupby(['key1', 'key2'])
df['col1'] = df['col1'] - g.transform('mean')
```

Suppose that you wanted to be able to demean more than one column and easily change the group keys. Additionally, you might want to perform this transformation in a method chain. Here is an example implementation:

```
def group_demean(df, by, cols):
    result = df.copy()
    g = df.groupby(by)
    for c in cols:
        result[c] = df[c] - g[c].transform('mean')
    return result
```

Then it is possible to write:

```
result = (df[df.col1 < 0]
          .pipe(group_demean, ['key1', 'key2'], ['col1']))
```





다음 주간에 배울 내용

Contents for Next Week

파이썬 모델링 라이브러리

- pandas 모델 인터페이스
- patsy
- statsmodel
- scikit-learn
- etc(통계 packages)



기획서 피드백 일정

Feedbacks for Proposal

1. 일요일에 나누어서 팀 전체별로 피드백 제공 예정.
2. 진행사항 관련해서 도움이 필요한 부분이 있다면,
자유롭게 언제든지 질문하시면 되겠습니다.
 - 간단한 수준에서는 평일 근무시간 혹은 퇴근시간 이후에도 상관없이 밤 11시까지 연락 가능
 - 주말에 물어보면 더 상세하게 시간을 내어 답변이 진행될 예정



Q&A

질의응답 (5분)



끝. 감사합니다.

수업 듣느라 수고하셨습니다.

